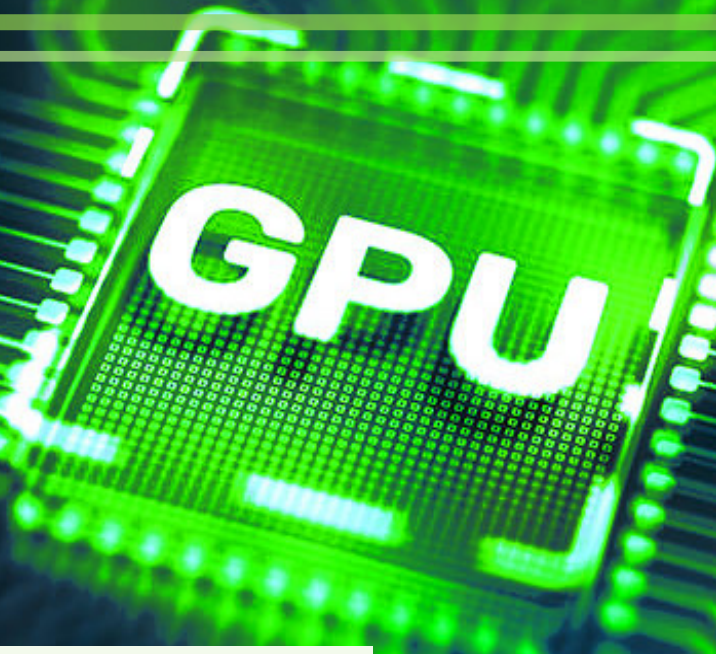


ARCHITECTURE ET PROGRAMMATION D'ACCÉLÉRATEURS MATÉRIELS

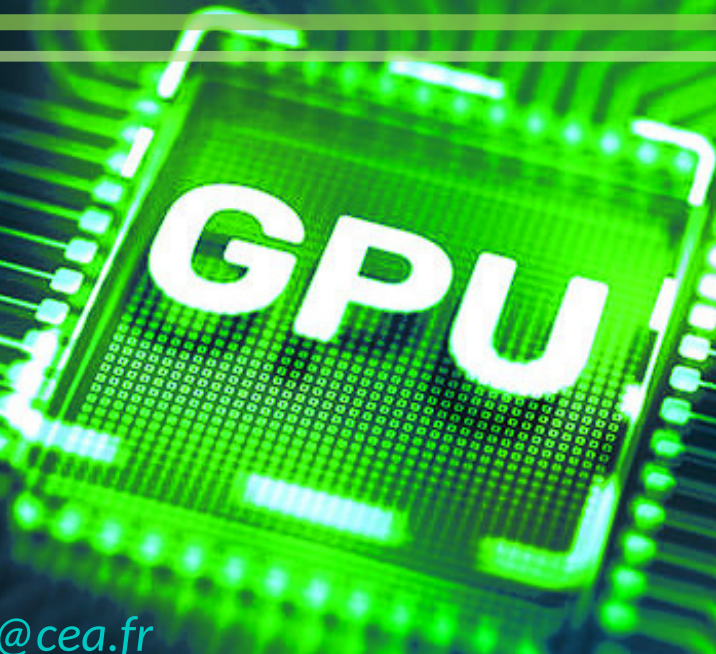
Lecture 1 : Architecture and programming models



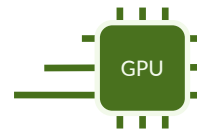
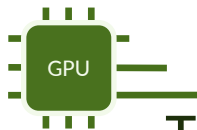
PROGRAMMATION GPU

GPU PROGRAMMING

Lecture 1 : Architecture and programming models



Introduction

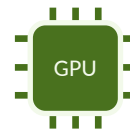
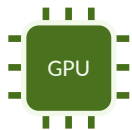


❖ Theme: *accelerators*

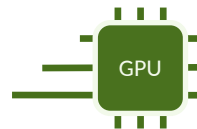
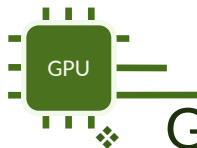
- ❖ State-of-the-art
- ❖ Accelerators architecture
- ❖ Accelerators programming
- ❖ Specific optimizations

❖ Prerequisites

- ❖ Basic notions in computer architecture
- ❖ Imperative programming (especially C)
- ❖ Notion in parallel programming



General outline



Goal :

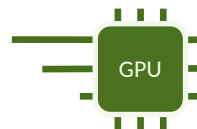
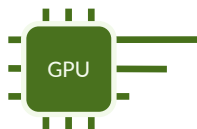
- ❖ Introduction to accelerators
- ❖ Expertise in CUDA programming

❖ Outline :

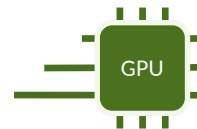
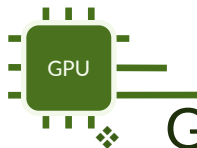
- ❖ Architecture
- ❖ Programming models
- ❖ Introduction to CUDA

❖ Teachers

- ❖ Julien Jaeger (julien.jaeger@cea.fr)
- ❖ Patrick Carribault (patrick.carribault@cea.fr)



General outline



Goal:

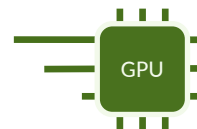
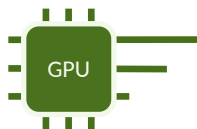
- ❖ Introduction to accelerators
- ❖ Expertise in CUDA programming

❖ Outline:

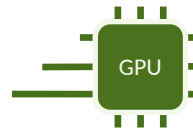
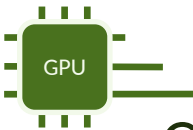
- ❖ Architecture
- ❖ Programming models
- ❖ Introduction to CUDA

❖ Teachers:

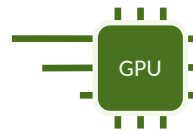
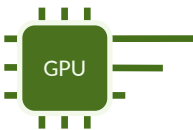
- ❖ Adrien Roussel (adrien.roussel@cea.fr)



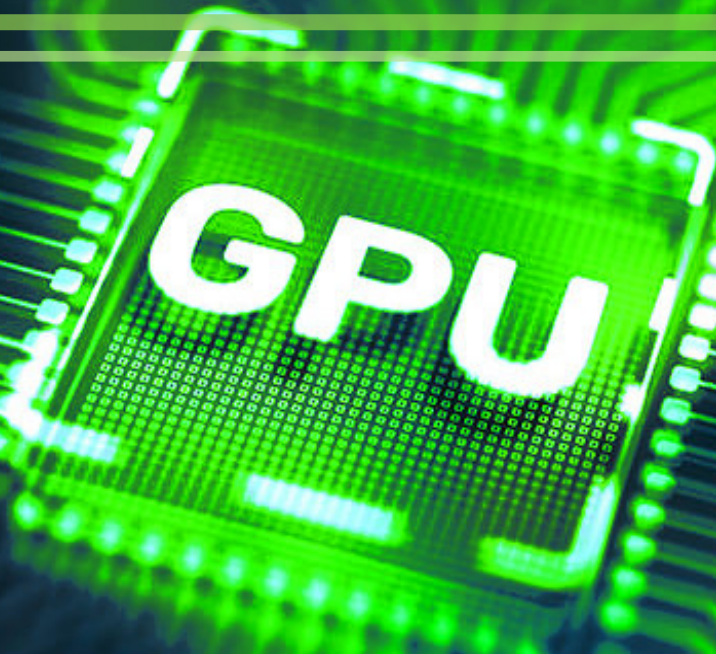
Lecture 1 Outline



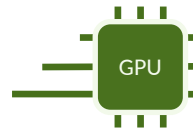
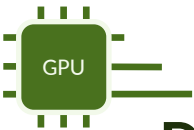
- ❖ Overview of existing accelerators
- ❖ Nvidia GPU architectures
 - ❖ NVIDIA Kepler
 - ❖ NVIDIA Volta
 - ❖ NVIDIA Ampere
 - ❖ NVIDIA Grace+Hopper
- ❖ Overview of programming models for accelerators
- ❖ NVIDIA CUDA (part 1)



OVERVIEW OF ACCELERATORS



Accelerators

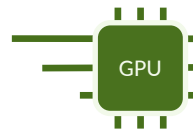
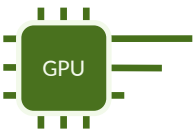


❖ Definition:

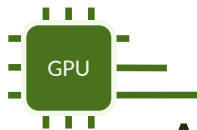
- ❖ A distinct resource (located on the same chip as the main processor or not) designed for different objectives than the central processor.

❖ In short:

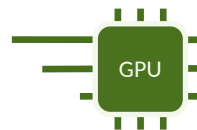
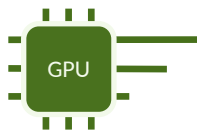
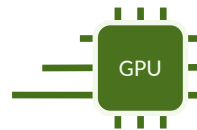
- ❖ A processor dedicated to a special class of applications.
- ❖ Allows for better performance in its domain of expertise.



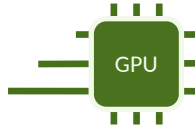
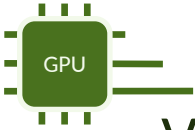
Accelerators – Interconnection



- ❖ Accelerators are attached to the processor(s) via an interconnect link.
- ❖ Behavior similar to a peripheral device:
 - ❖ The central processor controls access.
 - ❖ It also executes the main part of the code.
- ❖ Several types of interconnections:
 - ❖ Off-chip accelerators
 - ❖ On-chip accelerators

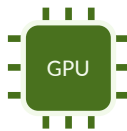
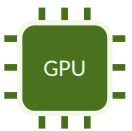


Types of connections

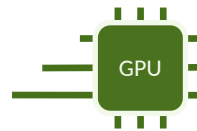
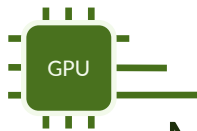


- ❖ Via the system bus:
 - ❖ Nvidia GPGPU, Intel MIC
- ❖ Advantages:
 - ❖ Low cost
 - ❖ Easy integration
- ❖ Drawbacks:
 - ❖ Modest performance due to the generic nature of the bus
 - ❖ Suboptimal bandwidth and latency

- ❖ Via the processor bus:
 - ❖ IBM Cell, AMD APU, ARM Grace + Nvidia Hopper
- ❖ Advantages:
 - ❖ Fast access
 - ❖ Possibility to share resources with the CPU
- ❖ Drawbacks:
 - ❖ Complex integration
 - ❖ Dependency on the processor



Accelerator Architecture



- ❖ **Microarchitecture:**

- ❖ Driven by the diversity of applications to be executed and the domain of expertise.
- ❖ Simplification compared to CPUs
 - ❖ (out-of-order execution, branch prediction, etc.).

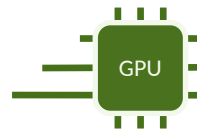
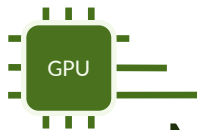
- ❖ **Exploited Parallelism:**

- ❖ Generally fine-grained.
- ❖ Most exploit a form of Single Instruction Multiple Data (SIMD).

- ❖ **But:**

- ❖ Significant part of the work is left to the software stack and developers.

Accelerator Architecture

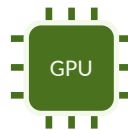
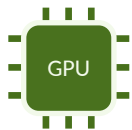


❖ Memory Hierarchy:

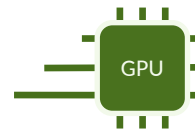
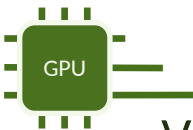
- ❖ Software-controlled memory (scratchpad memory).
- ❖ Hardware-managed cache (like a general-purpose CPU).
- ❖ Hardware interconnection between different compute processors.
- ❖ High bandwidth between the accelerator and global memory.

❖ Summary:

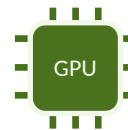
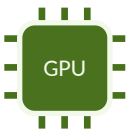
- ❖ Interesting possibilities for high-performance computing.
- ❖ But many aspects need to be managed manually.
- ❖ This directly influences how accelerators are programmed.



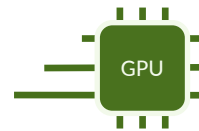
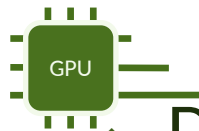
Accelerator Programming



- ❖ View of an accelerator as a worker peripheral.
- ❖ The main processor controls access to specialized computing resources:
 - ❖ Main processor (CPU): host
 - ❖ Accelerator: device
- ❖ General Concepts:
 - ❖ Data transfers
 - ❖ Execution of offloaded code in a different language
 - ❖ Manual execution management
 - ❖ Querying the accelerator for its status
 - ❖ Retrieving data back to the host

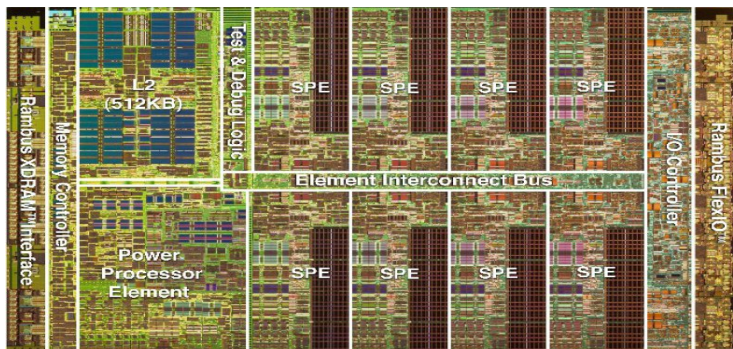
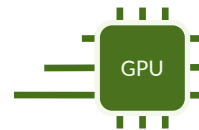
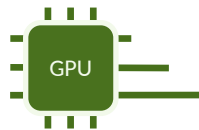


Overview of Accelerators



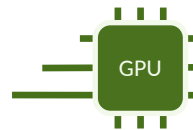
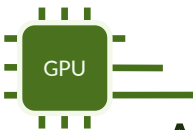
- ❖ Dedicated accelerators
 - ❖ Cell (IBM)
 - ❖ MIC or Xeon Phi (Intel)
- ❖ Graphics processors
 - ❖ NVIDIA
 - ❖ AMD/ATI
- ❖ Reconfigurables architectures
 - ❖ FPGA

CELL processor (IBM)

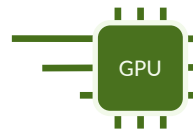
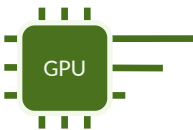


- ❖ Developed jointly by IBM, Sony, and Toshiba (2005+).
- ❖ 8 64-bit floating-point cores:
 - ❖ Called Synergistic Processor Elements (SPEs).
- ❖ One master 64-bit PowerPC processor capable of executing two threads.
- ❖ SPEs can process 128-bit operands, decomposed into four 32-bit words (SIMD).

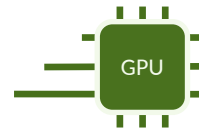
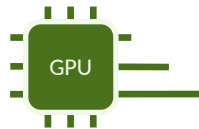
MIC & Xeon Phi (Intel)



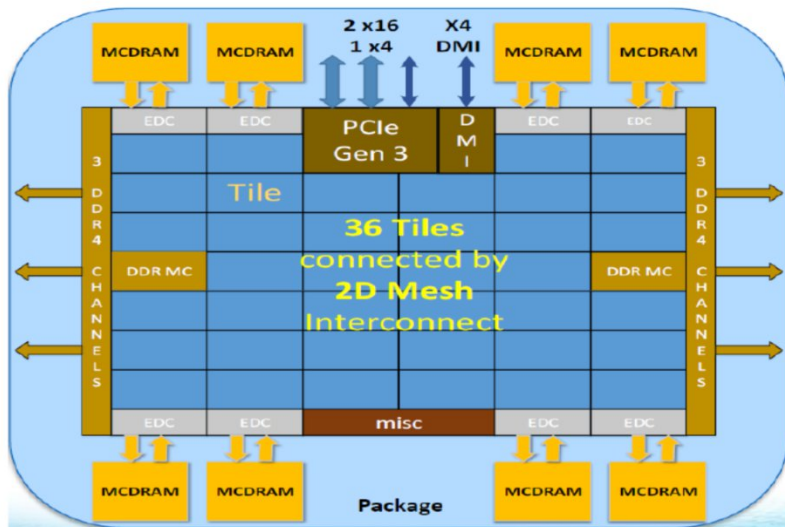
- ❖ Accelerator proposed by Intel:
 - ❖ Many-Integrated Core (MIC).
- ❖ Architecture:
 - ❖ x86 cores
 - ❖ Hyperthreading support
 - ❖ Cache coherence
- ❖ First version: KNF (2010)
- ❖ Latest version: KNL (2013) & KNM (2017)



KNL



KNL Overview



TILE

2 VPU	CHA	2 VPU
Core	1MB L2	Core

Chip: 36 Tiles interconnected by **2D Mesh**

Tile: 2 Cores + 2 VPU/core + 1 MB L2

Memory: MCDRAM: 16 GB on-package; High BW

DDR4: 6 channels @ 2400 up to 384 GB

IO: 36 lanes PCIe* Gen3. 4 lanes of DMI for chipset

Node: 1-Socket only

Fabric: Intel® Omni-Path Architecture on-package (not shown)

Vector Peak Perf: 3+TF DP and 6+TF SP Flops

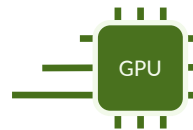
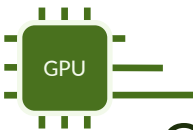
Scalar Perf: ~3x over Knights Corner

Streams Triad (GB/s): MCDRAM : 400+; DDR: 90+

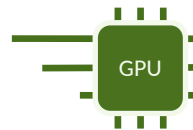
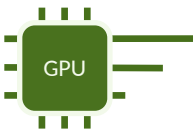
MCDRAM
~5X Higher BW
than DDR

- Source Intel: All products, computer systems, dates and figures specified are preliminary based on current expectations, and are subject to change without notice. KNL data are preliminary based on current expectations and are subject to change without notice. 1Binary Compatible with Intel Xeon processors using Haswell Instruction Set (except TSX). 2Bandwidth numbers are based on STREAM-like memory access pattern when MCDRAM used as flat memory. Results have been estimated based on internal Intel analysis and are provided for informational purposes only. Any difference in system hardware or software design or configuration may affect actual performance. *Other names and brands may be claimed as the property of others.

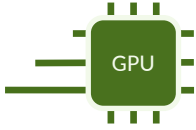
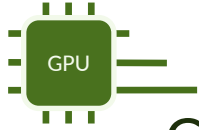
Graphics Processors



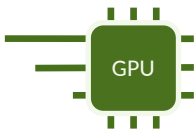
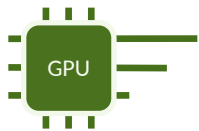
- ❖ Graphics cards initially for 2D and 3D display on screens:
 - ❖ Dedicated graphics processor(s).
- ❖ Impressive computing capabilities:
 - ❖ Optimized for image rendering (different objectives from general-purpose processors).
 - ❖ Ideal for processing regular data.
- ❖ Use for scientific computing:
 - ❖ GPGPU (General Purpose Graphics Processing Unit).



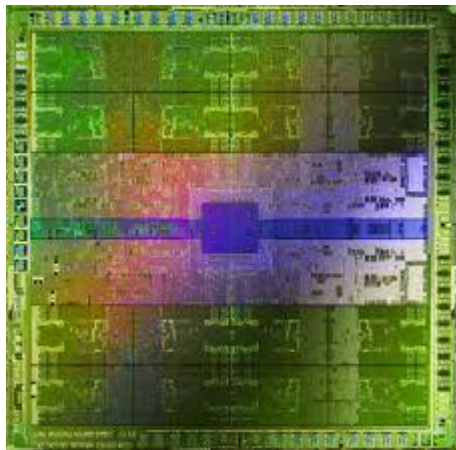
GPGPU



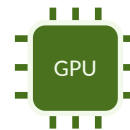
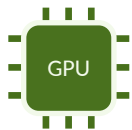
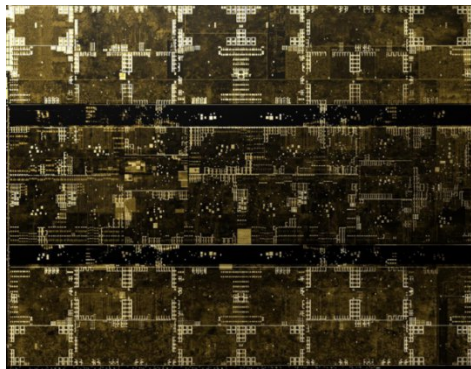
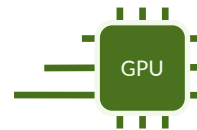
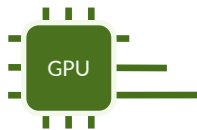
- ❖ Graphics cards on a system bus:
 - ❖ Interconnection on the motherboard.
 - ❖ Communications via the PCI-express port.
- ❖ Possibility to connect multiple graphics cards:
 - ❖ Multi-GPU programming.
 - ❖ Currently used in heterogeneous supercomputers.



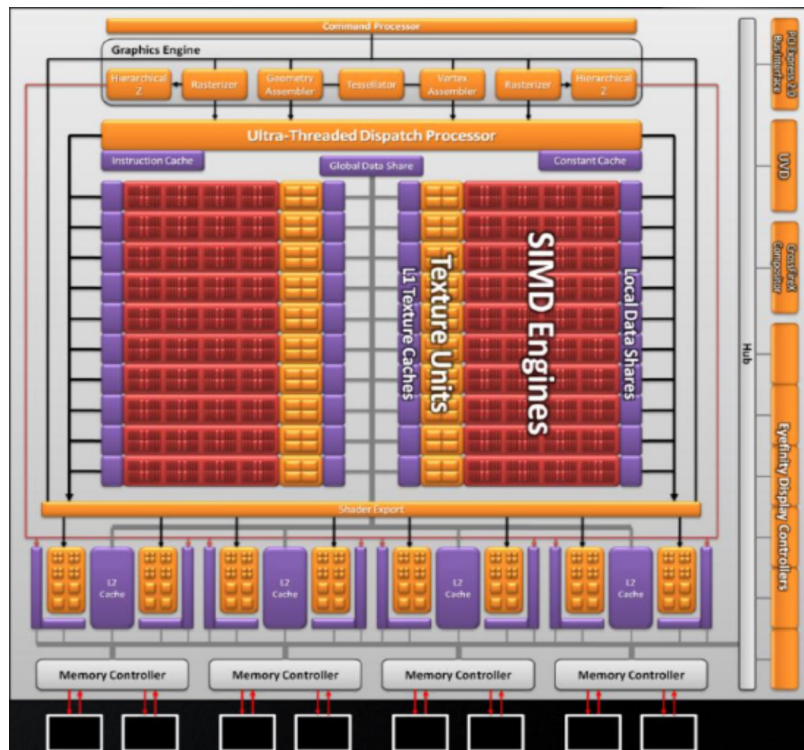
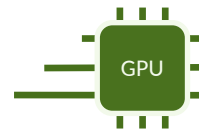
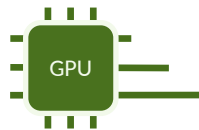
Nvidia Fermi Architecture (2010)



Nvidia Volta Architecture (2017)

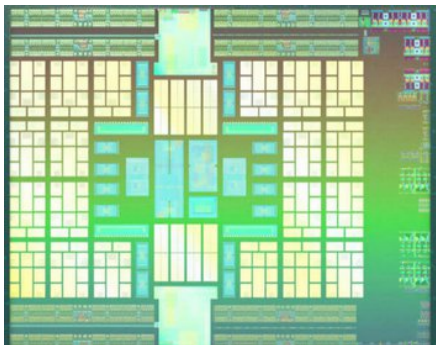


ATI Cypress Architecture (2009)

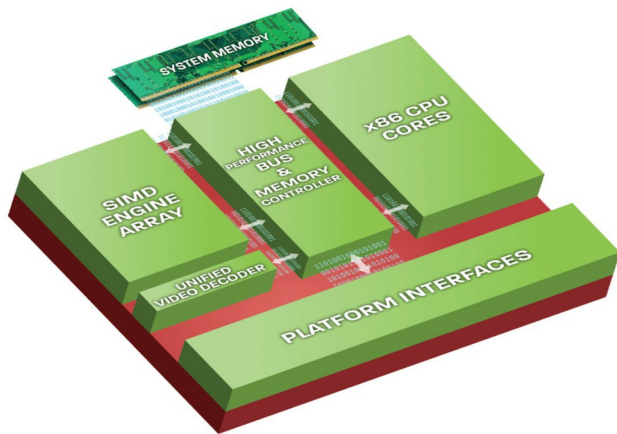
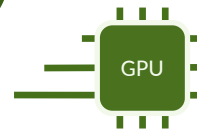
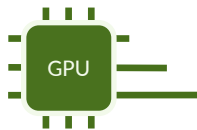


- ❖ Set of vector units:
 - ❖ SIMD Engines.
- ❖ Memory hierarchy:
 - ❖ L2 Cache.
- ❖ Peak performance:
 - ❖ 2.72 TFlops SP
 - ❖ 0.54 TFlops DP

ATI Radeon Navi Architecture (2019)

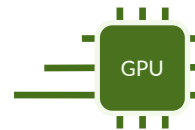
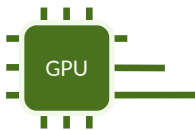


AMD Fusion Llano Architecture (2011)

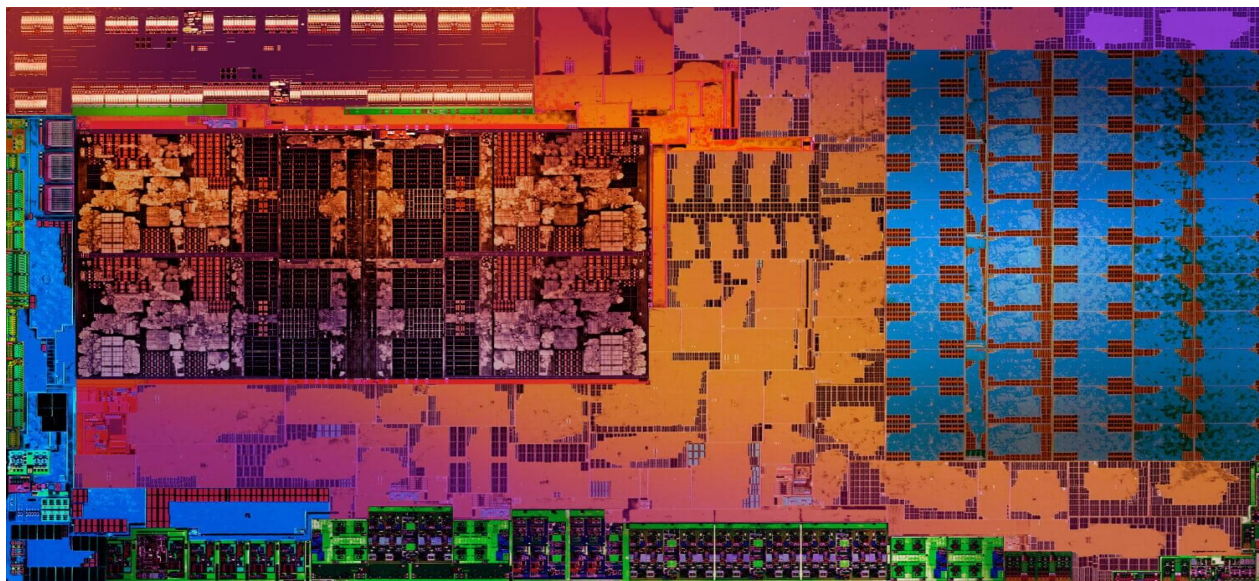


- ❖ Nom de code de l'architecture d'AMD/ATI : Fusion
- ❖ Design d'un APU (Accelerator Processing Unit)
- ❖ Sur la même puce :
 - ❖ CPU
 - ❖ Accélérateur de type GPU
- ❖ Première génération disponible en 2011
 - ❖ Llano avec technologie Bulldozer ou Bobcat
- ❖ <http://fusion.amd.com>

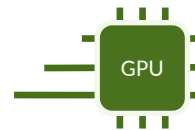
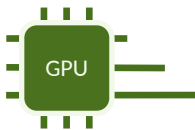
AMD APU Picasso Architecture



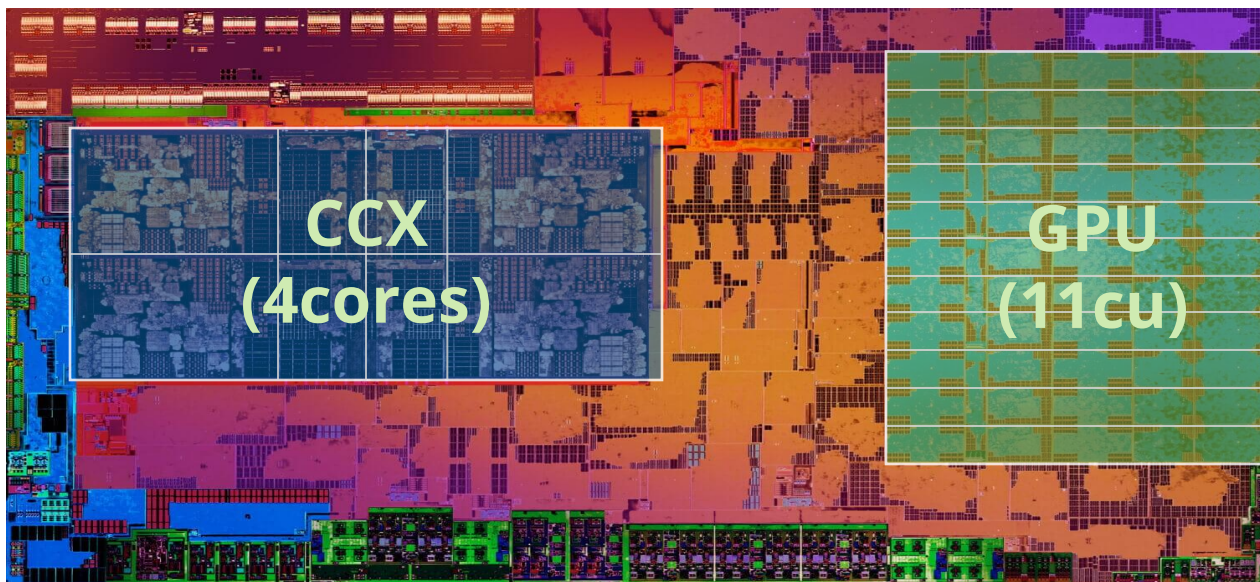
■ Ryzen-bridge 2019



AMD APU Picasso Architecture



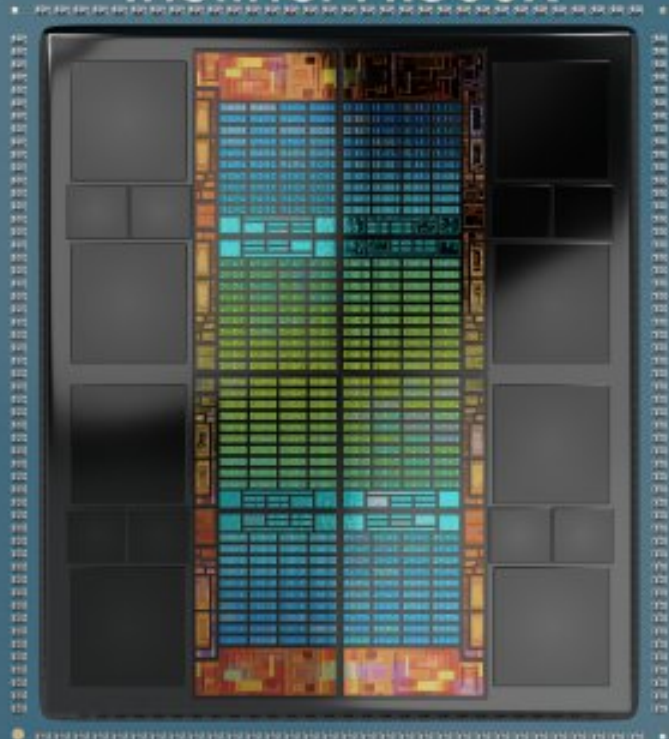
■ Ryzen-bridge 2019



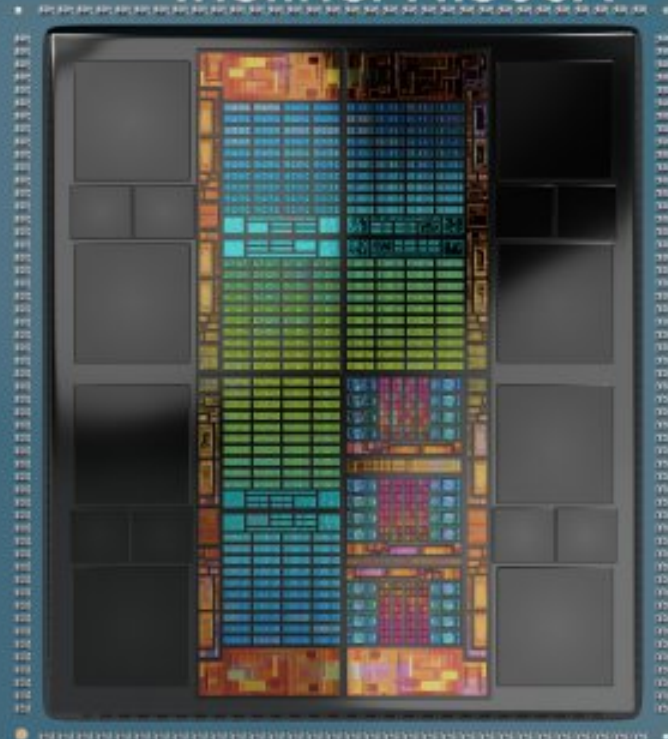
AMD MI300 – Frontier (2023)

GPU

Instinct MI300X

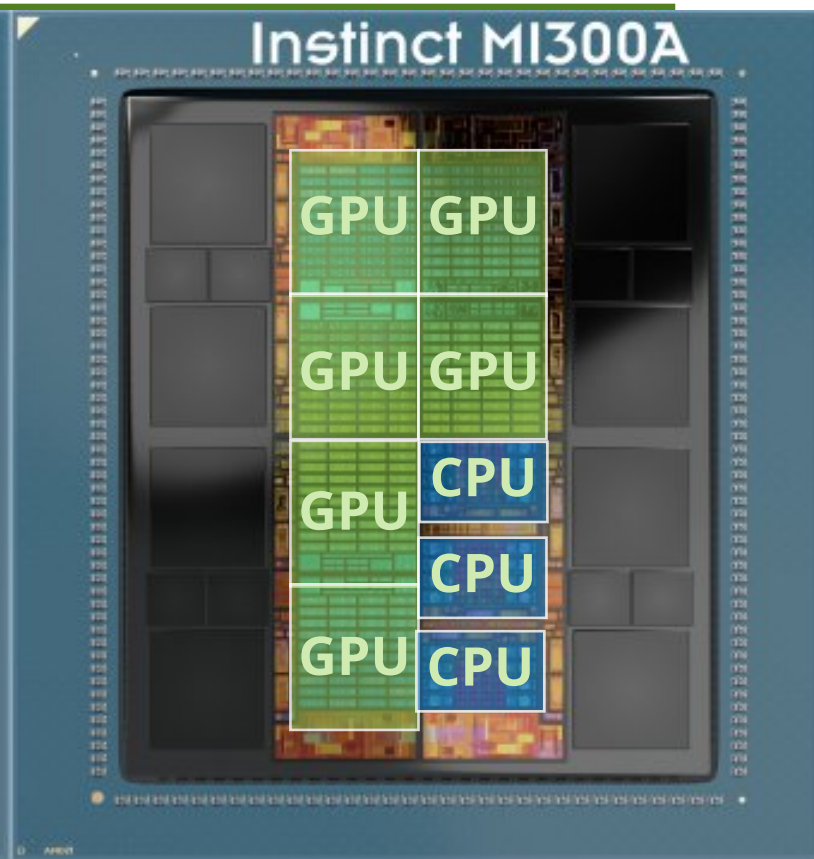
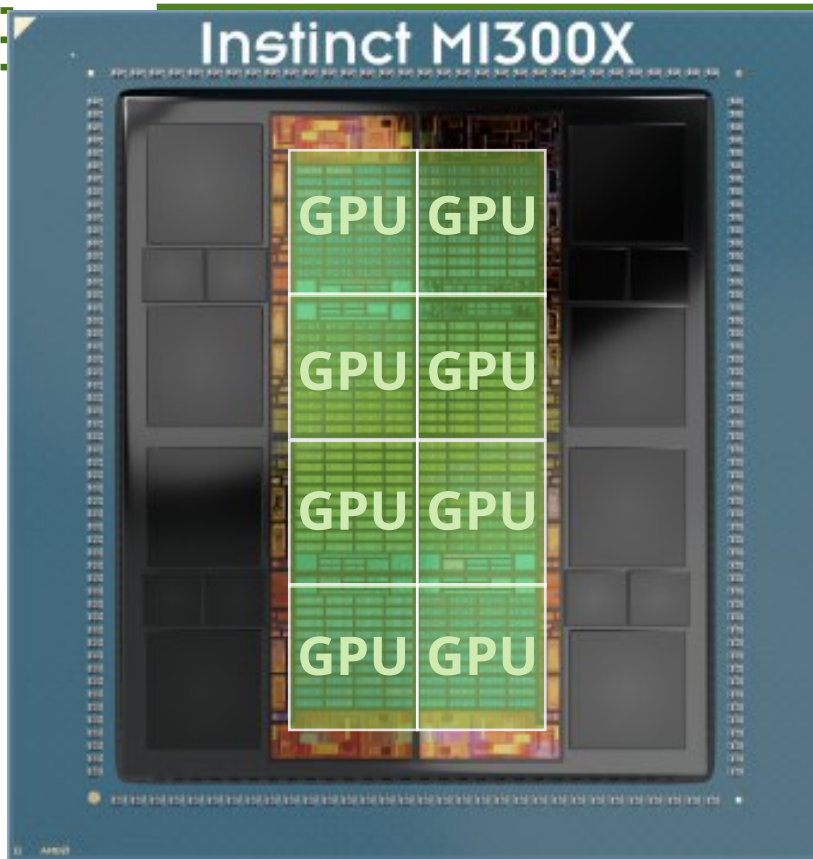
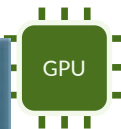
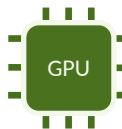


Instinct MI300A

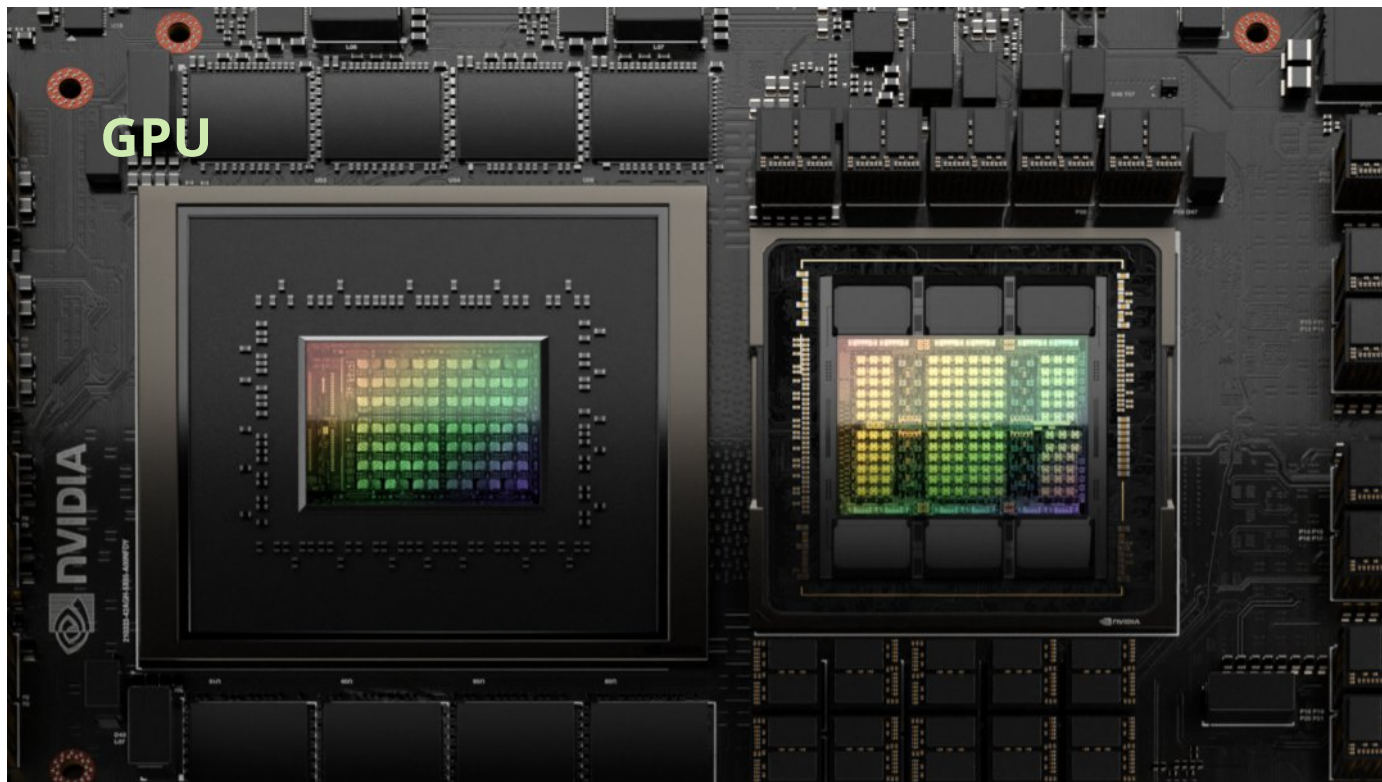
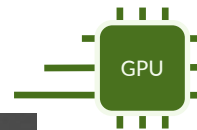
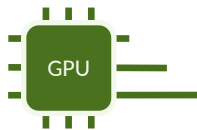


GPU

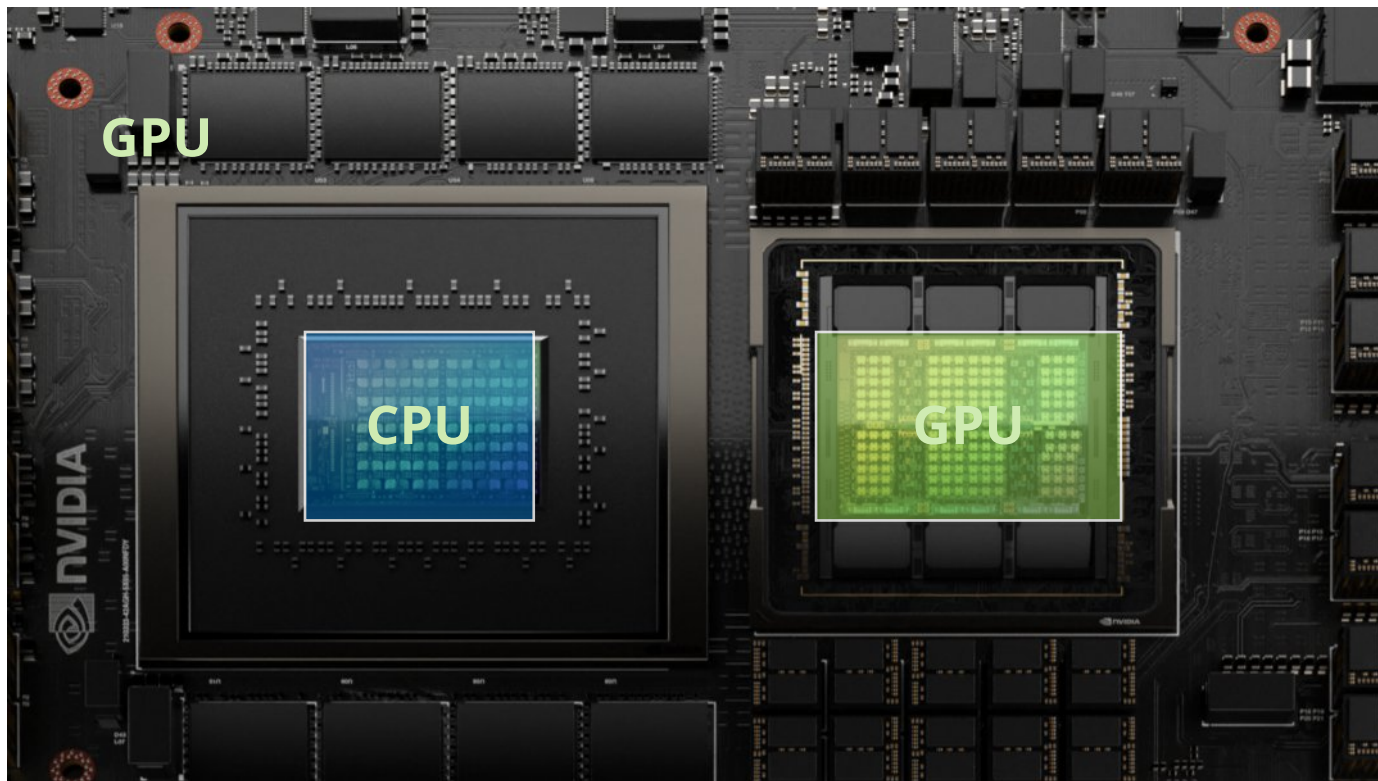
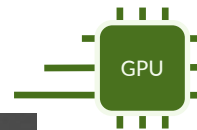
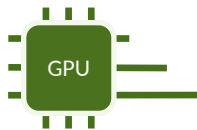
AMD MI300 – Frontier (2023)



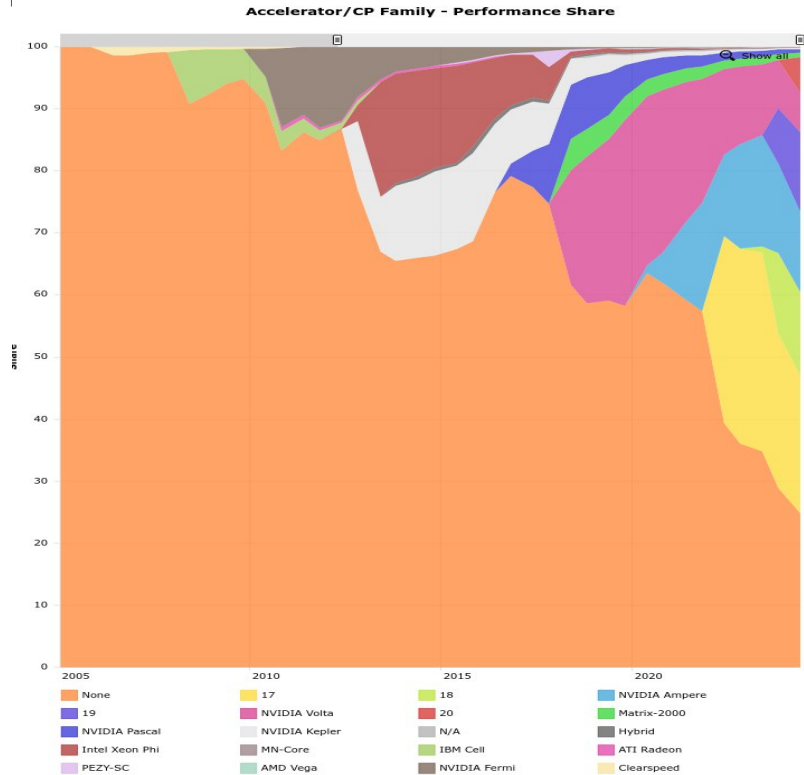
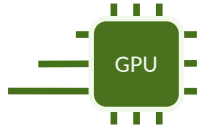
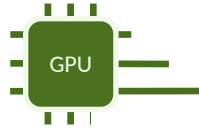
Nvidia Grace+Hopper (2022)



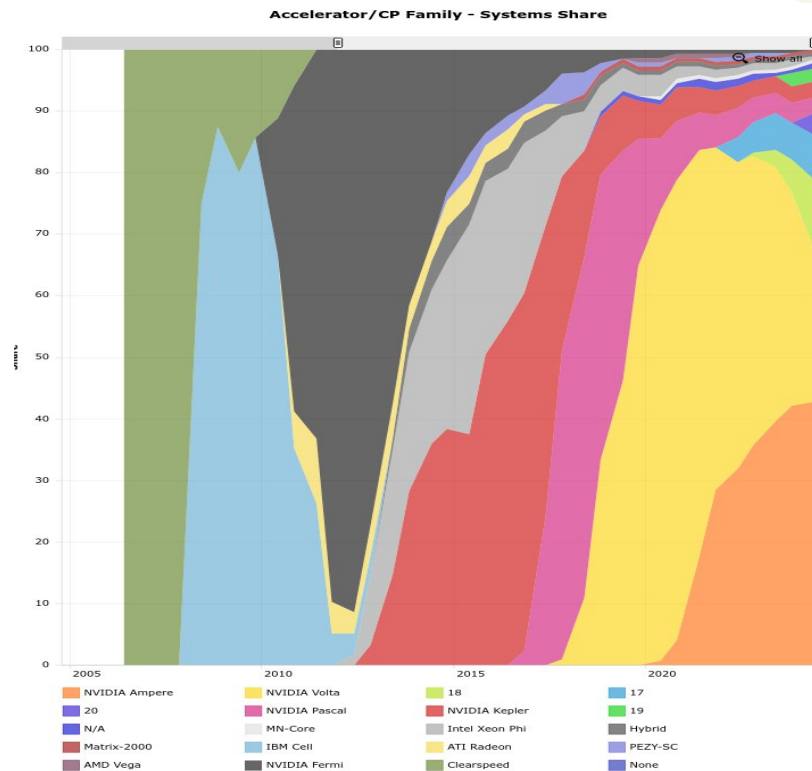
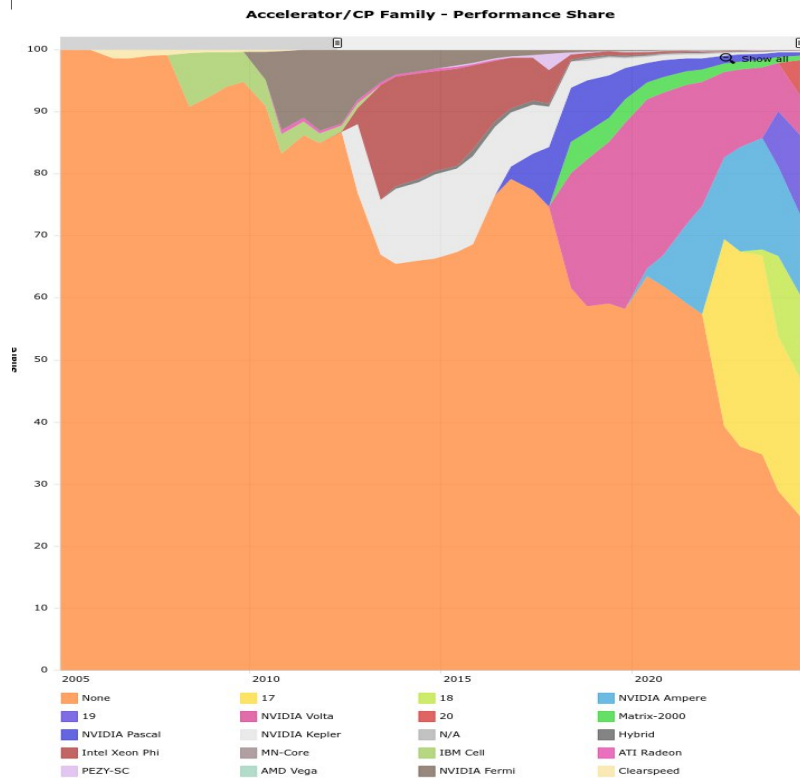
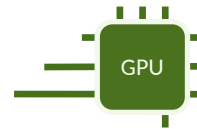
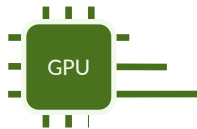
Nvidia Grace+Hopper (2022)



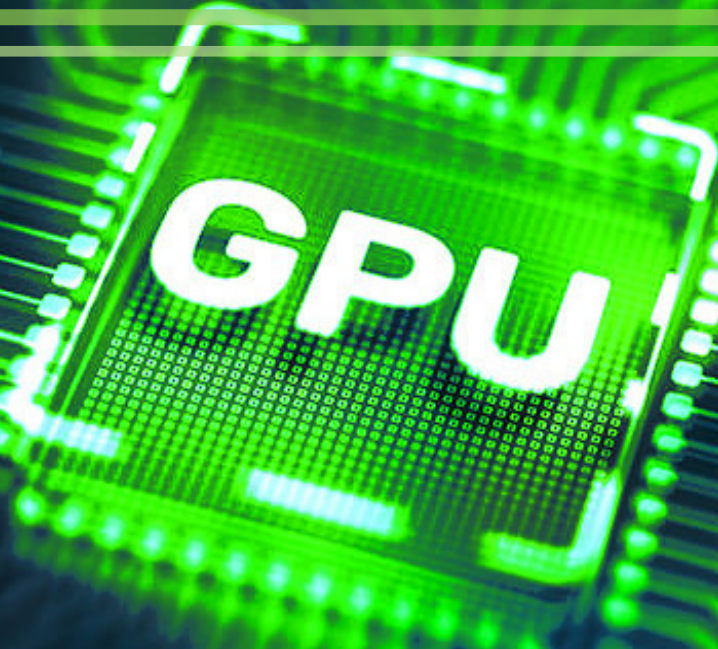
Evolution of accelerators in the Top500



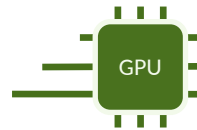
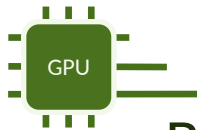
Evolution of accelerators in the Top500



NVIDIA GRAPHICS CARD ARCHITECTURE

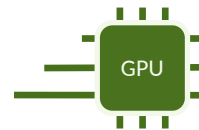
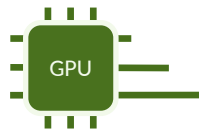


NVIDIA GeForce Architecture



- ❖ Basic Building Block:
 - ❖ Very simple compute core.
 - ❖ Set of compute cores grouped in a stream multiprocessor.
- ❖ Why "stream"?
 - ❖ Designed to operate on a continuous data stream.
 - ❖ Synchronous language on each multicore.
- ❖ Design for Fine-Grained Parallelism

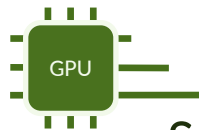
Nvidia Kepler Architecture (2012)



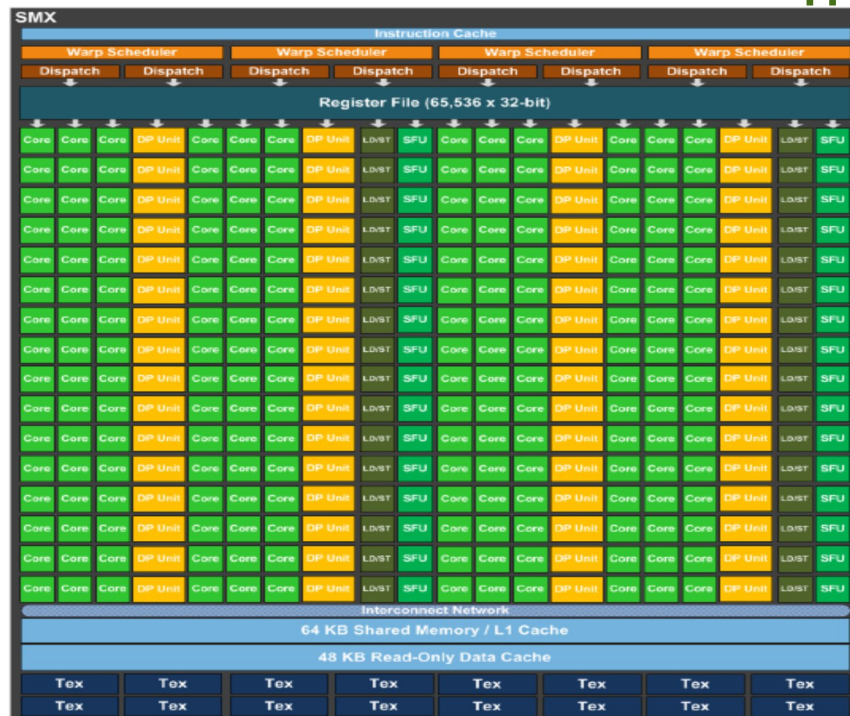
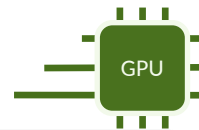
- ❖ 15 Streaming Multiprocessors (SMs).
- ❖ Arrangement perpendicular to the shared L2 cache.
- ❖ **Legend:**
 - ❖ Green: Compute core (double precision unit in orange).
 - ❖ Orange: Scheduler and dispatcher.
 - ❖ Light blue: Register bank and L1 cache.



Kepler : Streaming Multiprocessor



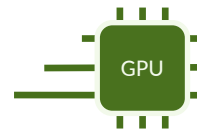
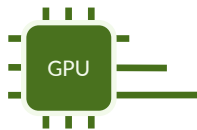
- ❖ Streaming Multiprocessor (SMX).
- ❖ 192 CUDA-compatible cores:
 - ❖ ALU unit.
 - ❖ Single-precision floating-point unit.
- ❖ 64 double-precision floating-point units.
- ❖ 32 special units (SFU).
- ❖ 32 load/store units.



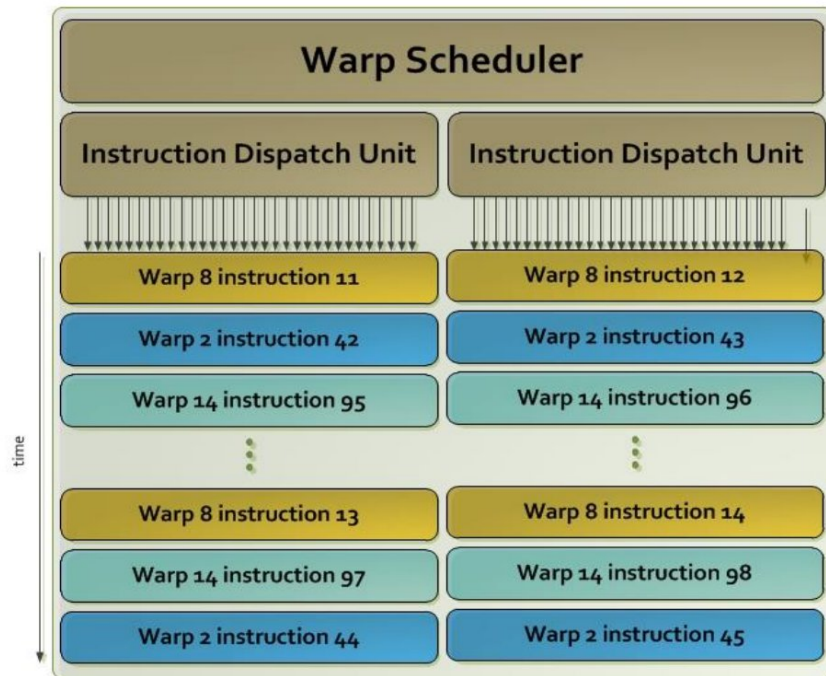
- 



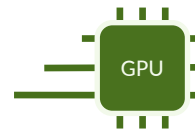
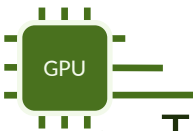
Kepler : Warp Scheduler



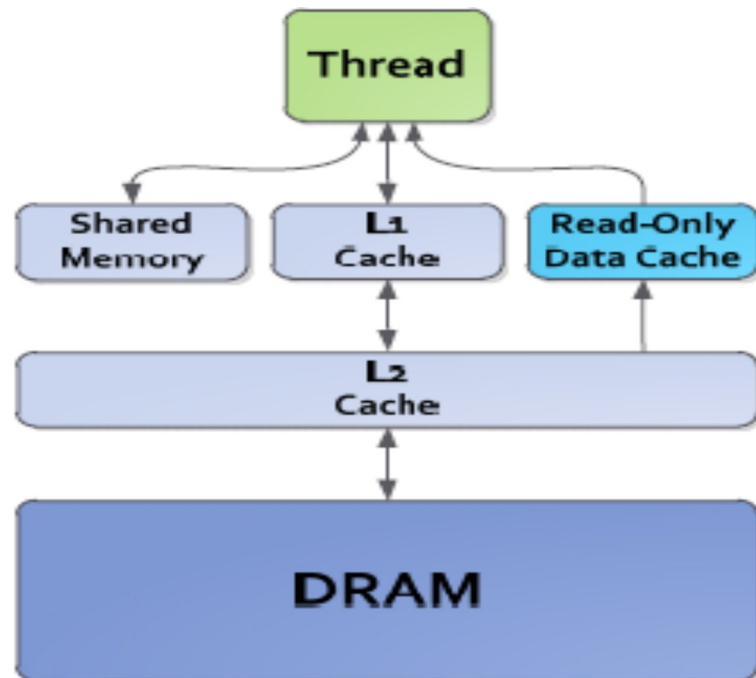
- ❖ 4 Warp Scheduler / SMX
 - ❖ 1 Warp = 32 threads
- ❖ 8 Dispatch Unit / SMX
 - ❖ 2 Dispatch Unit / Warp Scheduler
- ❖ Quad Warp Scheduler – How does it work ?
 - ❖ Select 4 Warps (1 Warp / Warp Scheduler)
 - ❖ 2 independent instructions per Warp may be scheduled every cycles.



Kepler: Memory Hierarchy



- ❖ Thread scheduled on a compute core on an SMX.
- ❖ Memory access via load or store instruction.
- ❖ Two possibilities:
 - ❖ Access to shared memory: no cache.
 - ❖ Access to global device memory: passes through two levels of cache.
- ❖ Physically, the same memory is used for L1 cache and shared memory:
 - ❖ Appearance of a read-only cache (48 KB).

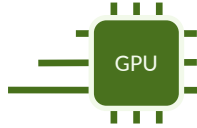
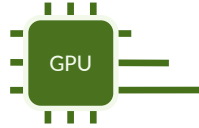


Nvidia Volta (2017)



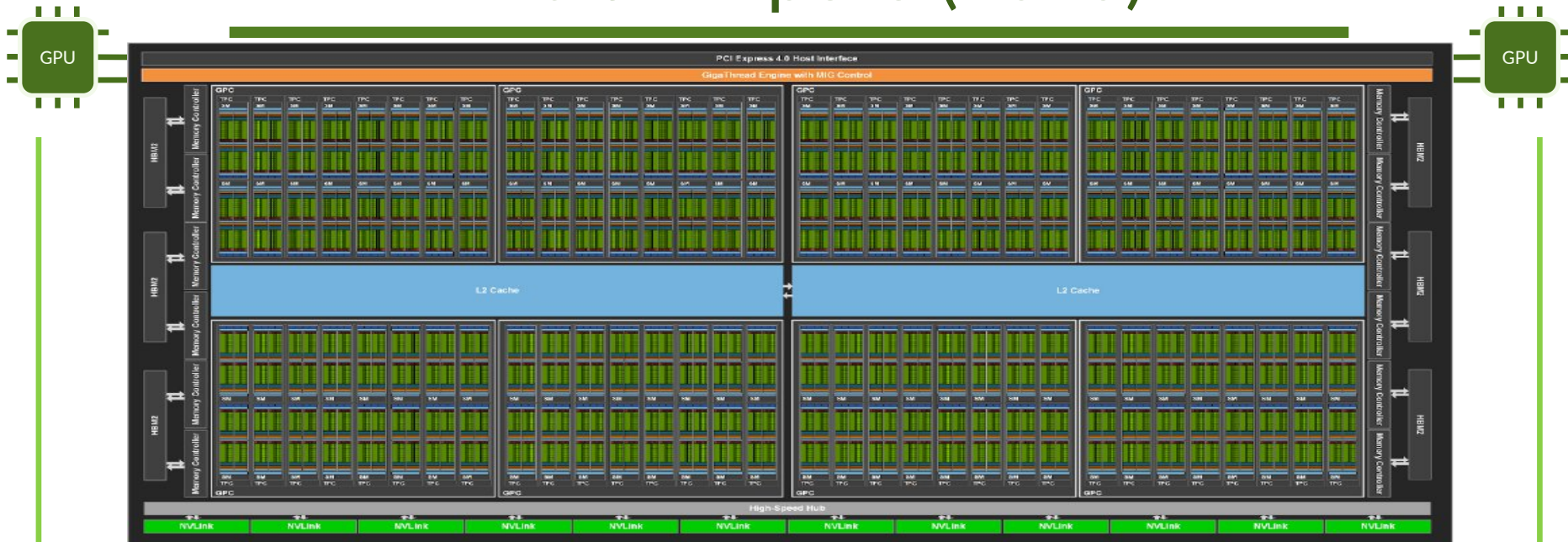
- ❖ 6 GPCs (Graphics Processing Clusters).
- ❖ 7 TPCs (Texture Processing Clusters) per GPC.
- ❖ 2 SMs per TPC.
- ❖ Total: $2 * 7 * 6 = 84$ SMs!

Nvidia Volta (2017)



- ❖ 5376 single-precision floating-point units (14 Tflops) and same amount of integer units.
- ❖ 2688 double-precision floating-point units (7 Tflops).
- ❖ 672 tensor cores (112 Tflops).
- ❖ + 336 texture units.

Nvidia Ampere (2020)



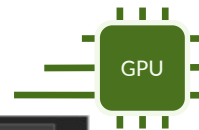
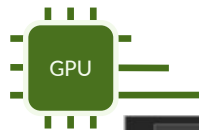
- ❖ 8 GPCs (Graphics Processing Clusters).
- ❖ 8 TPCs (Texture Processing Clusters) per GPC.
- ❖ 2 SMs per TPC.
- ❖ Total: $2 * 8 * 8 = 128$ SMs!

Nvidia Ampere: SM view

- ❖ 4 processing blocks per SM.
- ❖ 16 INT units: 8192 (21 Tops).
- ❖ 16 single-precision floating-point units: 8192 (21 Tflops).
- ❖ 8 double-precision floating-point units: 4096 (10 Tflops).
- ❖ 1 tensor core (112 Tflops).



Nvidia Hopper (2023)



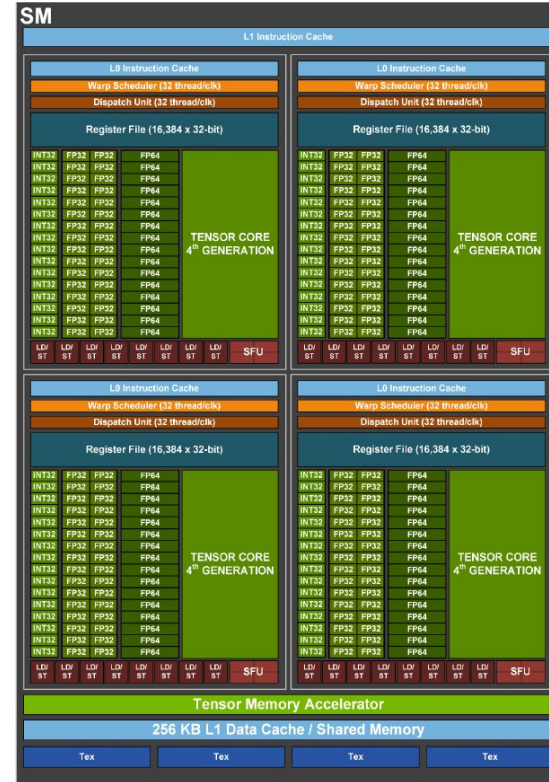
- ❖ 8 GPCs (Graphics Processing Clusters).
- ❖ 9 TPCs (Texture Processing Clusters) per GPC.
- ❖ 2 SMs per TPC.
- ❖ Total: $2 * 9 * 8 = 144$ SMs!

Hopper : SM view

GPU

- ❖ 4 × Processing Blocks:
- ❖ 16 INT32 units.
- ❖ 32 single-precision units.
- ❖ 16 double-precision units.
- ❖ 8 load/store units.
- ❖ 1 warp scheduler.
- ❖ 1 dispatch unit.
- ❖ Maximum 64 active warps.

GPU



GPU

GPU

Hopper Performance

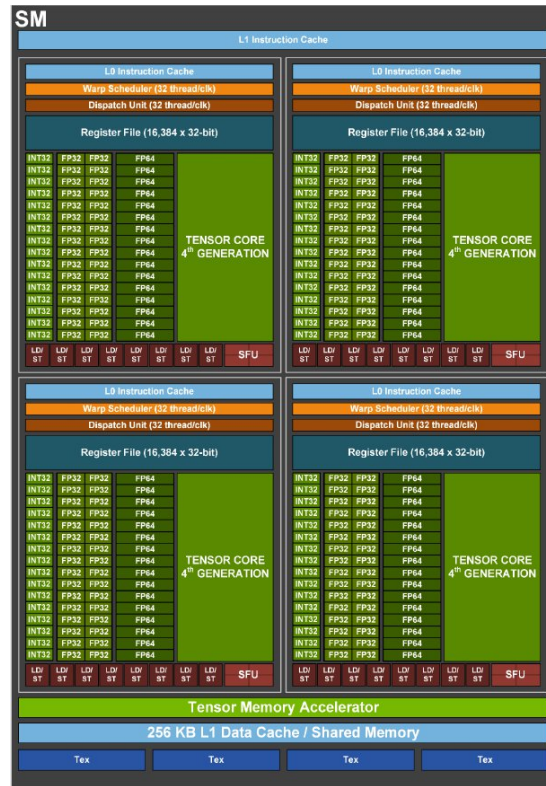
GPU

- ❖ "Classical" computing:
 - ❖ FP32: 60 Tflops.
 - ❖ FP64: 30 Tflops.
- ❖ "AI" computing (tensor cores):
 - ❖ FP64: 60 Tflops.
 - ❖ TF32: 1 Pflops.
 - ❖ FP16: 2 Pflops.
 - ❖ FP8: 4 Pflops.
- ❖ Difference between FP32 and TF32:
 - ❖ FP32: 8-bit exponent/23-bit mantissa.
 - ❖ TF32: 8-bit exponent/10-bit mantissa.
 - ❖ FP16: 5-bit exponent/10-bit mantissa.

GPU

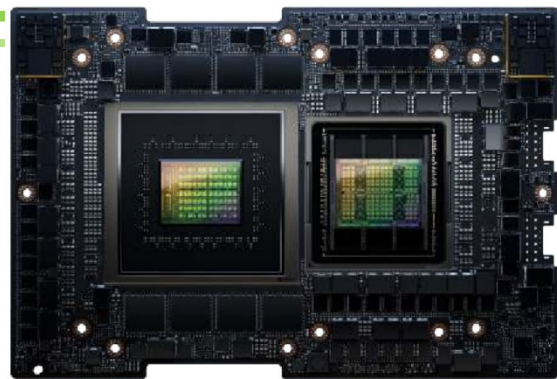
GPU

GPU

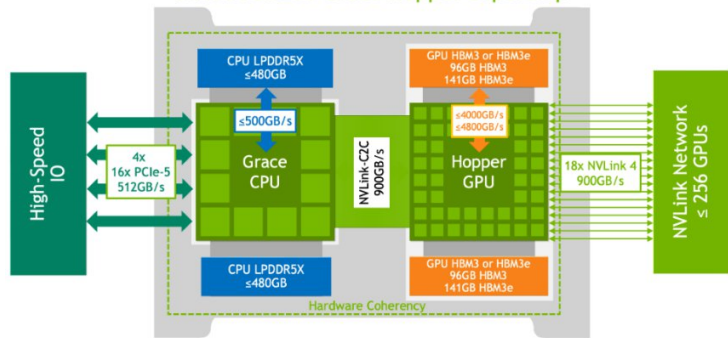


Grace-Hopper Superchip: The Future?

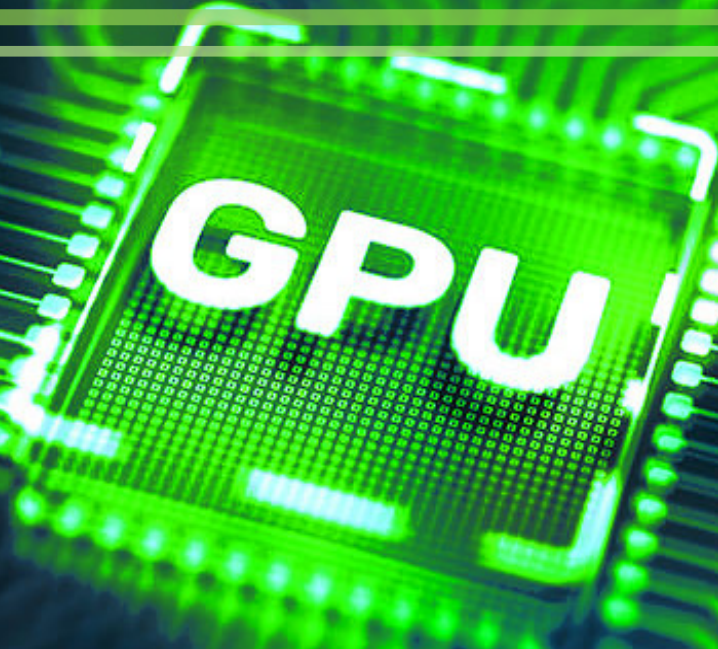
- ❖ "Grace" processor:
 - ❖ ARM Neoverse V2 architecture.
 - ❖ 72 cores.
- ❖ CPU and GPU on the same die to accelerate data transfers:
 - ❖ 2x 450GB/s.
 - ❖ Native unified memory guaranteed by a memory coherence system (C2C).
- ❖ Beware of NUMA effects!



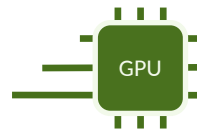
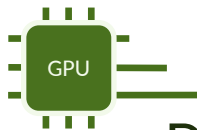
NVIDIA GH200 Grace Hopper Superchip



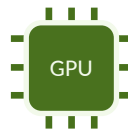
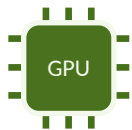
PROGRAMMING MODELS FOR ACCELERATORS



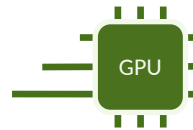
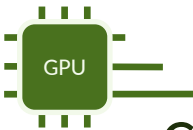
Programming Models



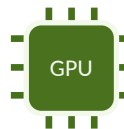
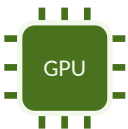
- ❖ Dedicated Languages:
 - ❖ Close To Metal (CTM), HIP/ROCm, CUDA.
- ❖ Portable Languages:
 - ❖ Low-level languages
 - ❖ OpenCL.
 - ❖ Directives based:
 - ❖ OpenMP, OpenACC.
 - ❖ High-Level Languages / DSLs:
 - ❖ Kokkos, Legion, RAJA, SYCL



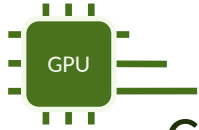
ATI/AMD Programming



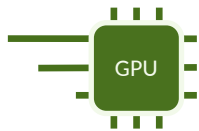
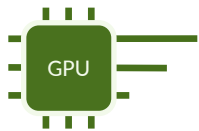
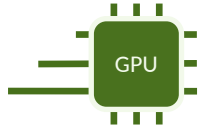
- ❖ Close to Metal (CTM) / CAL/IL:
 - ❖ Very low level.
 - ❖ Rarely used now.
- ❖ HIP/ROCm:
 - ❖ Provides a toolkit for writing GPU code and translating CUDA code to HIP.
- ❖ OpenCL/SYCL:
 - ❖ Supports OpenCL and SYCL standards.
 - ❖ AMD/ATI is part of the consortium.



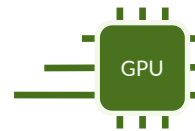
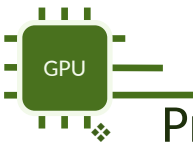
NVIDIA Programming



- ❖ CUDA Language:
 - ❖ To be continued...



Directive based



Proposal for directive-based programming:

- ❖ Adding #pragma (C, C++) in sequential code.
- ❖ Possibility to add these directives in already parallel code (MPI, OpenMP, etc.): depends on the model.

❖ Proposed solutions:

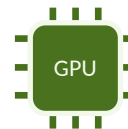
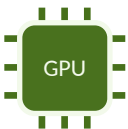
- ❖ OpenACC
- ❖ OpenMP 4+

❖ Advantages:

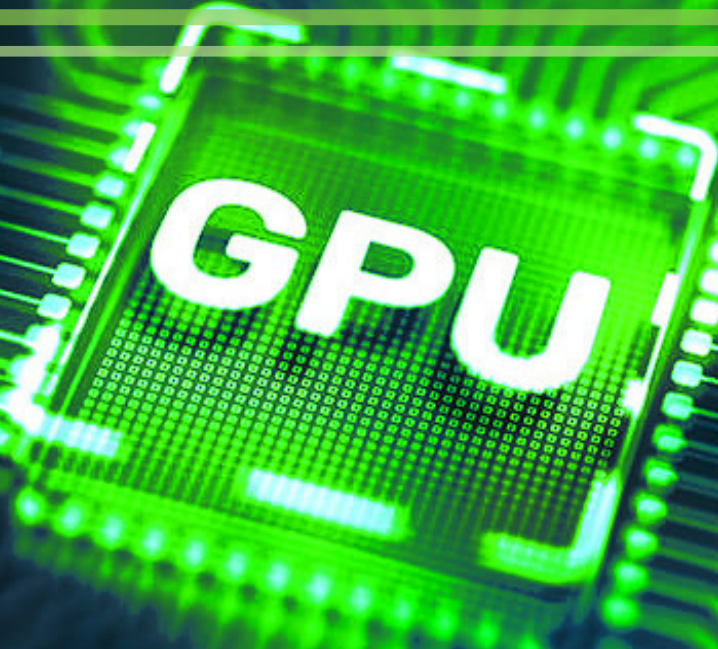
- ❖ Portability depending on the accelerators.
- ❖ Original (or even sequential) semantics can be preserved.

❖ Drawbacks:

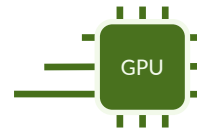
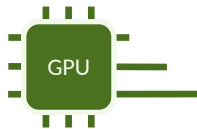
- ❖ Often need to finish "by hand".
- ❖ Sometimes limited scope.



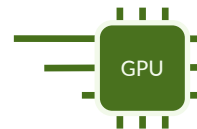
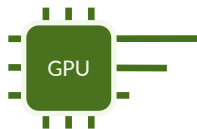
NVIDIA CUDA



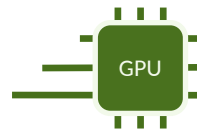
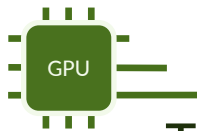
According to you...



- ❖ What do you think we will find in a GPU program?
 - ❖ Which parts?
 - ❖ Which steps?

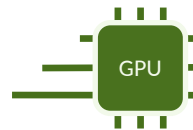
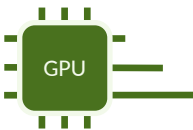


Overview

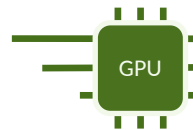
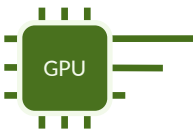


- ❖ Two parts: host program and compute kernels.
- ❖ Need to transfer data between hosts and devices.
- ❖ Dedicated compilation chain due to compute kernels (different language).
- ❖ Link with a runtime (user space) and a driver (kernel space).

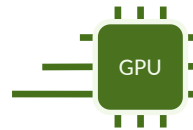
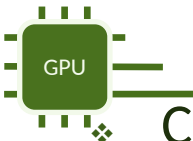
Programming Scheme



1. Initialization on host:
 - ❖ Memory allocation.
 - ❖ Reading inputs.
 - ❖ ...
2. Memory allocation on the GPU:
 - ❖ Manual management!
3. Transfer host → GPU.
4. Execution of a compute kernel:
 - ❖ Possibility to launch multiple kernels.
5. Transfer GPU → host.
6. Free GPU memory.
7. Free CPU memory.

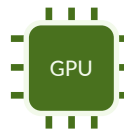
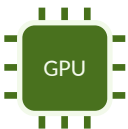


How to compile?



Code Structure:

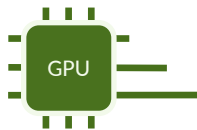
- ❖ Part for the host.
- ❖ Part for the device.
- ❖ In practice, these codes can be mixed in the same file.
- ❖ Host Programming:
 - ❖ C or C++.
- ❖ Device Programming:
 - ❖ CUDA (superset of C99 standard).
- ❖ Compilation Chain: NVCC:
 - ❖ As a programmer: use the nvcc compiler.
 - ❖ Do not hesitate to check the documentation provided with the SDK.



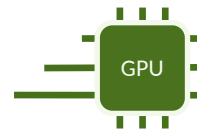
Example Program

- ❖ Example: Adding 2 vectors.
- ❖ Basic operation:
 - ❖ $c[i] = a[i] + b[i]$.
- ❖ How to program this compute kernel sequentially?
 - ❖ Simple loop.
 - ❖ Apply the basic operation to each element.

```
void vectAdd(  
    double * a, double * b,  
    double * c, int N) {  
    int i ;  
    for (i=0;i<N;i++) {  
        c[i] = a[i] + b[i]  
    }  
}
```

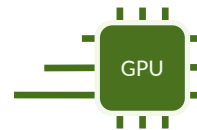
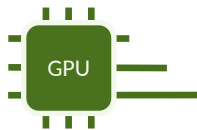


Porting Our Example



```
void vectAdd(  
    double * a, double * b,  
    double * c, int N) {  
    int i ;  
    for (i=0;i<N;i++) {  
        c[i] = a[i] + b[i]  
    }  
}
```

Porting Our Example



```
void vectAdd(  
    double * a, double * b,  
    double * c, int N) {  
    int i ;  
    for (i=0;i<N;i++) {  
        c[i] = a[i] + b[i]  
    }  
}
```

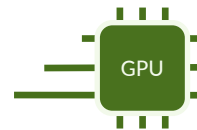
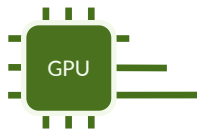
Transfert du
tableau c

Transfert du
tableau a

Transfert du
tableau b

Transfert de
l'entier N

Porting Our Example



```
void vectAdd(  
    double * a, double * b,  
    double * c, int N) {  
    int i ;  
    for (i=0;i<N;i++) {  
        c[i] = a[i] + b[i]  
    }  
}
```

Transfert du
tableau c

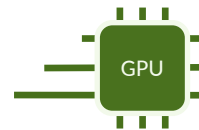
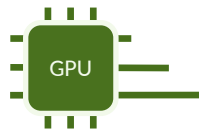
Transfert du
tableau a

Transfert du
tableau b

Transfert de
l'entier N

Exécution sur le
GPU
(en parallèle ??)

Porting Our Example



```
void vectAdd(  
    double * a, double * b,  
    double * c, int N) {  
    int i ;  
    for (i=0;i<N;i++) {  
        c[i] = a[i] + b[i]  
    }  
}
```

Transfert du
tableau c

Exécution sur le
GPU
(en parallèle ??)

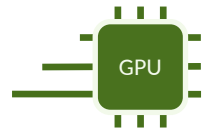
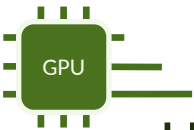
Transfert du
tableau a

Transfert du
tableau b

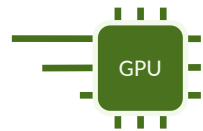
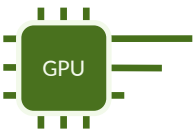
Transfert de
l'entier N

Rapatriement du
tableau c

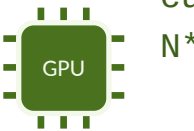
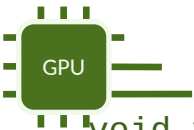
Host Programming



- ❖ Host (Central CPU) Management:
 - ❖ Need to manage GPU memory.
 - ❖ Need to transfer data.
 - ❖ Gives the order to execute the kernel.
- ❖ CUDA Host API:
 - ❖ High-level/low-level CUDA.
 - ❖ `cudaMalloc`: Allocate data on the device.
 - ❖ `cudaMemcpy`: Transfer data to the device or to the host.

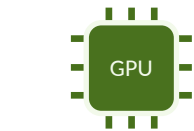
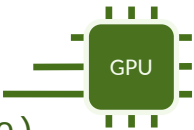


Host program



```
void vectAdd(  
    double * a, double * b, double * c,  
    int N) {  
  
    double * d_a ;  
    double * d_b ;  
    double * d_c ;  
  
    cudaMalloc((void **)&d_a,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_b,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_c,  
        N*sizeof(double));
```

```
        cudaMemcpy(d_a, a, N*sizeof(double),  
            cudaMemcpyHostToDevice);  
        cudaMemcpy(d_b, b, N*sizeof(double),  
            cudaMemcpyHostToDevice);  
  
        vectAddKernel(d_a, d_b, d_c, N);  
  
        cudaMemcpy(c, d_c, N*sizeof(double),  
            cudaMemcpyDeviceToHost);  
  
        cudaFree(d_a);  
        cudaFree(d_b);  
        cudaFree(d_c);
```



Host program

GPU

GPU

```
void vectAdd(  
    double * a, double * b, double * c,  
    int N) {
```

```
    double * d_a ;  
    double * d_b ;  
    double * d_c ;
```

Device
pointers

```
    cudaMalloc((void **)&d_a,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_b,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_c,  
        N*sizeof(double));
```

```
    cudaMemcpy(d_a, a, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
        cudaMemcpyHostToDevice);
```

```
    vectAddKernel(d_a, d_b, d_c, N);
```

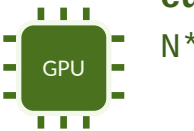
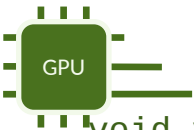
```
    cudaMemcpy(c, d_c, N*sizeof(double),  
        cudaMemcpyDeviceToHost);
```

```
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);
```

GPU

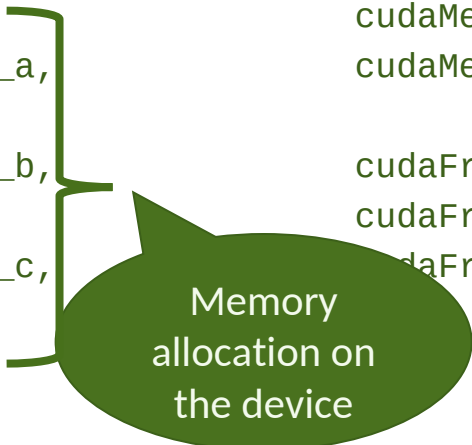
GPU

Host program



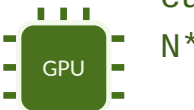
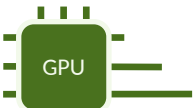
```
void vectAdd(  
    double * a, double * b, double * c,  
    int N) {  
  
    double * d_a ;  
    double * d_b ;  
    double * d_c ;  
  
    cudaMalloc((void **)&d_a,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_b,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_c,  
        N*sizeof(double));
```

```
        cudaMemcpy(d_a, a, N*sizeof(double),  
            cudaMemcpyHostToDevice);  
        cudaMemcpy(d_b, b, N*sizeof(double),  
            cudaMemcpyHostToDevice);  
  
        vectAddKernel(d_a, d_b, d_c, N);  
  
        cudaMemcpy(c, d_c, N*sizeof(double),  
            cudaMemcpyDeviceToHost);  
  
        cudaFree(d_a);  
        cudaFree(d_b);  
        cudaFree(d_c);
```



Memory
allocation on
the device

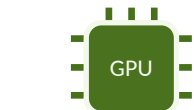
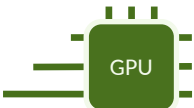
Host program



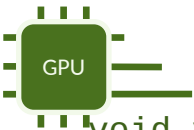
```
void vectAdd(  
    double * a, double * b, double * c,  
    int N) {  
  
    double * d_a ;  
    double * d_b ;  
    double * d_c ;  
  
    cudaMalloc((void **)&d_a,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_b,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_c,  
        N*sizeof(double));
```

Data transfers
from host to
device

```
        cudaMemcpy(d_a, a, N*sizeof(double),  
            cudaMemcpyHostToDevice);  
        cudaMemcpy(d_b, b, N*sizeof(double),  
            cudaMemcpyHostToDevice);  
  
    vectAddKernel(d_a, d_b, d_c, N);  
  
    cudaMemcpy(c, d_c, N*sizeof(double),  
        cudaMemcpyDeviceToHost);  
  
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);
```



Host program



```
void vectAdd(  
    double * a, double * b, double * c,  
    int N) {
```

```
    double * d_a ;  
    double * d_b ;  
    double * d_c ;
```

Kernel invocation
(will execute on
the device)

```
    cudaMalloc((void **)&d_a,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_b,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_c,  
        N*sizeof(double));
```

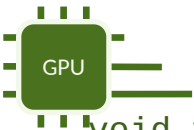
```
    cudaMemcpy(d_a, a, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
        cudaMemcpyHostToDevice);
```

```
    vectAddKernel(d_a, d_b, d_c, N);
```

```
    cudaMemcpy(c, d_c, N*sizeof(double),  
        cudaMemcpyDeviceToHost);
```

```
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);
```

Host program



```
void vectAdd(  
    double * a, double * b, double * c,  
    int N) {
```

```
    double * d_a ;  
    double * d_b ;  
    double * d_c ;
```

```
    cudaMalloc((void **)&d_a,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_b,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_c,  
        N*sizeof(double));
```

Data transfer
from device to
host

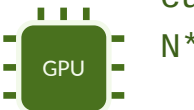
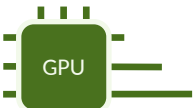
```
    cudaMemcpy(d_a, a, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
        cudaMemcpyHostToDevice);
```

```
    vectAddKernel(d_a, d_b, d_c, N);
```

```
    cudaMemcpy(c, d_c, N*sizeof(double),  
        cudaMemcpyDeviceToHost);
```

```
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);
```

Host program



```
void vectAdd(  
    double * a, double * b, double * c,  
    int N) {  
  
    double * d_a ;  
    double * d_b ;  
    double * d_c ;  
  
    cudaMalloc((void **)&d_a,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_b,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_c,  
        N*sizeof(double))
```

```
        cudaMemcpy(d_a, a, N*sizeof(double),  
            cudaMemcpyHostToDevice);  
        cudaMemcpy(d_b, b, N*sizeof(double),  
            cudaMemcpyHostToDevice);
```

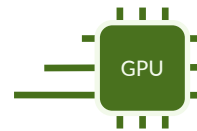
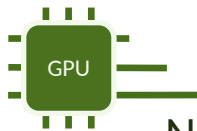
```
        vectAddKernel(d_a, d_b, d_c, N);
```

```
        cudaMemcpy(c, d_c, N*sizeof(double),  
            cudaMemcpyDeviceToHost);
```

```
        cudaFree(d_a);  
        cudaFree(d_b);  
        cudaFree(d_c);
```

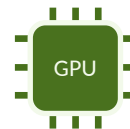
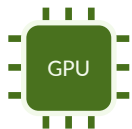
Free memory
allocated on
device

Execution model

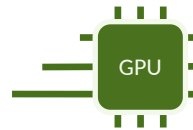
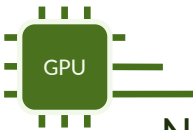


- ❖ Now remains to execute the kernel on the device
- ❖ How to program the following code on the *device* ?

```
for (i=0;i<N;i++)  
    c[i]=a[i]+b[i];
```



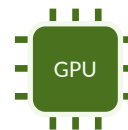
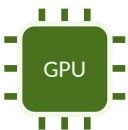
Execution model



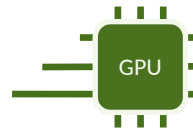
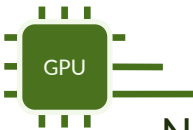
- ❖ Now remains to execute the kernel on the device
- ❖ How to program the following code on the *device* ?

```
for (i=0;i<N;i++)  
    c[i]=a[i]+b[i];
```

- ❖ The CUDA model is a model based on threads
 - ❖ At invocation of a kernel, launches a bunch of threads
 - ❖ Each thread executes the *kernel*
 - ❖ Quite similar to passing the kernel function to several `pthread_create` invocations
- ❖ CUDA programming is *multithreaded*
 - ❖ Need to think per thread, and not per loop or loop iteration



Execution model

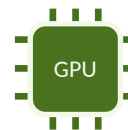
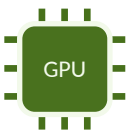


- ❖ Now remains to execute the kernel on the device
- ❖ How to program the following code on the *device* ?

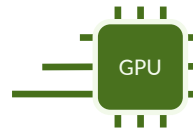
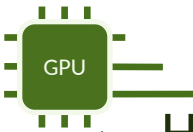
```
for (i=0;i<N;i++)  
    c[i]=a[i]+b[i];
```

- ❖ The CUDA model is a model based on threads
 - ❖ At invocation of a kernel, launches a bunch of threads
 - ❖ Each thread executes the *kernel*
 - ❖ Quite similar to passing the kernel function to several `pthread_create` invocations
- ❖ CUDA programming is *multithreaded*
 - ❖ Need to think per thread, and not per loop or loop iteration
- ❖ Code du *kernel* dans notre cas :

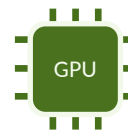
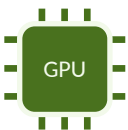
```
int i = index; /* Update i with thread index */  
c[i] = a[i] + b[i];
```



Threads hierarchy

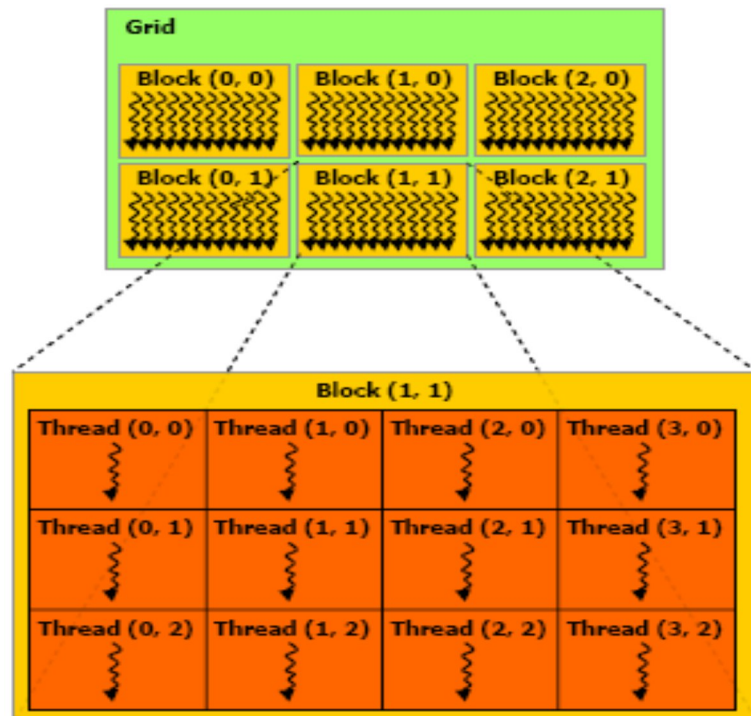


- ❖ How to Know the Identifier of a Thread?
 - ❖ This is related to the organization of threads within the CUDA execution model.
- ❖ CUDA defines the concepts of blocks and grids to hierarchically organize threads:
 - ❖ A grid contains blocks.
 - ❖ A block contains threads.
- ❖ Both grids and blocks can have 1 to 3 dimensions (logical representation, not physical).
- ❖ In Practice ?



Threads hierarchy

- ❖ Example with
 - ❖ A grid with blocks organized in 2D
 - ❖ 6 blocks with threads also in 2D
 - ❖ 6 blocks in the grid with 12 threads per block
 - 72 threads in total
 - ❖ How to find a thread index ?
 - ❖ 2D array linearization



Threads hierarchy

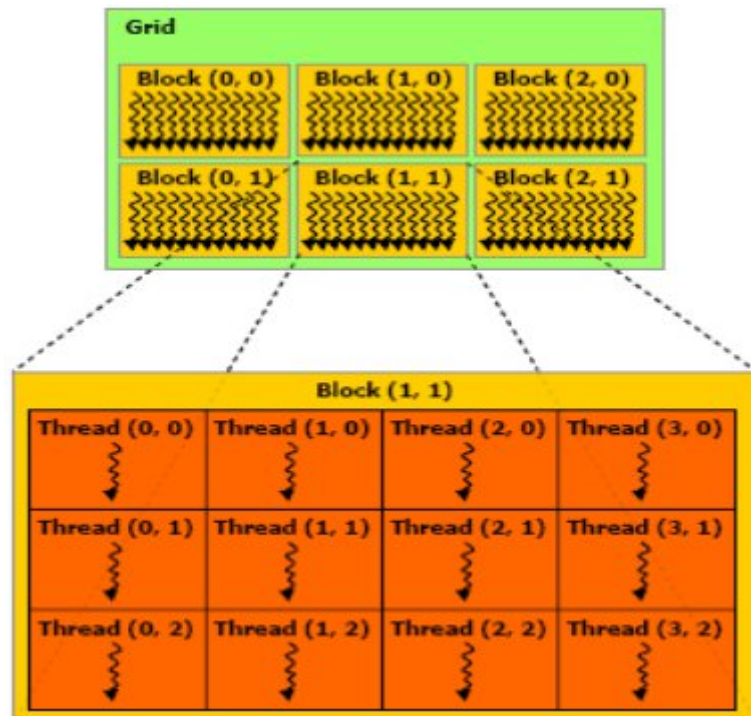
GPU

GPU

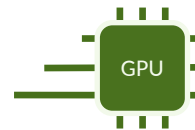
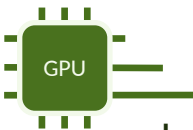
CUDA defines several variables in the kernel

- ❖ **gridDim** : Contains the number of blocks in each dim
- ❖ **blockIdx** : Contains the coordinates of the block in the grid for the three dimensions
- ❖ **blockDim** : Contains the number of threads in each dim
- ❖ **threadIdx** : Contains the coordinates of the thread in its block for the three dimensions
- ❖ The type of these variables is a CUDA specific type:
 - ❖ dim3
 - ❖ It has three fields : {x, y, z} for values on each dimension

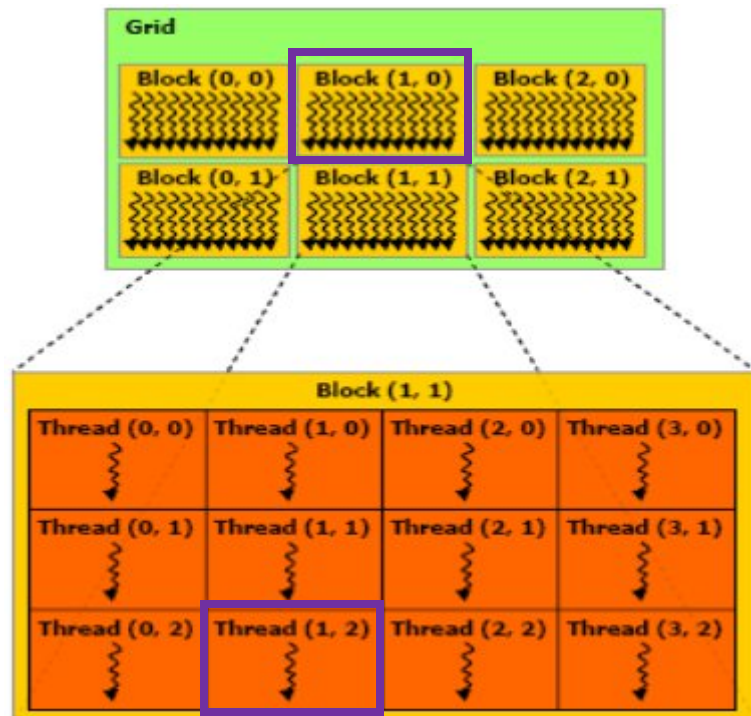
GPU



Threads hierarchy



Let's compute the index of the chosen thread



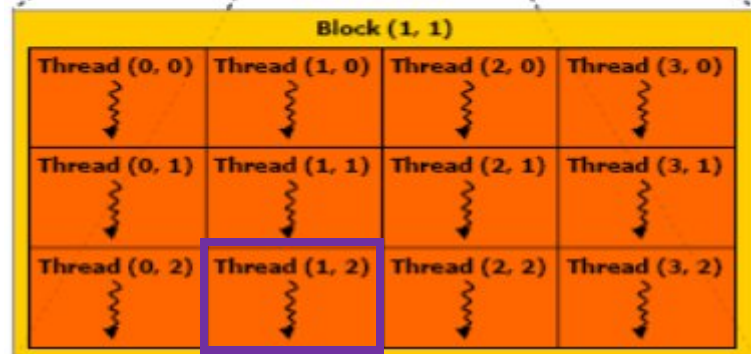
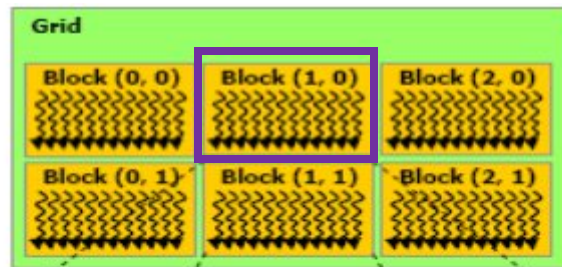
Threads hierarchy

GPU

GPU

Let's compute the index of the chosen thread

- ❖ $\text{BlockIdx.x} = 1$; $\text{BlockIdx.y} = 0$
- ❖ $\text{GridDim.x} = 3$; $\text{GridDim.y} = 2$



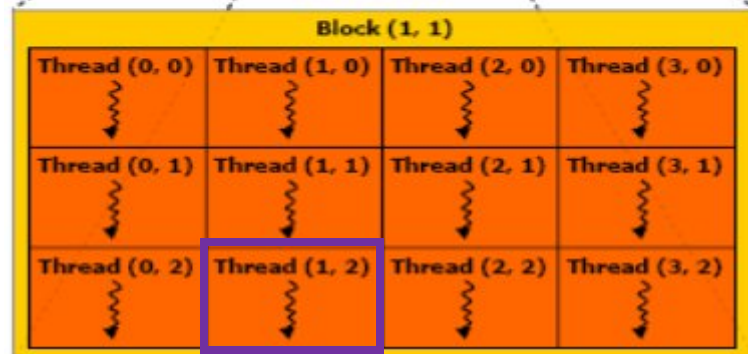
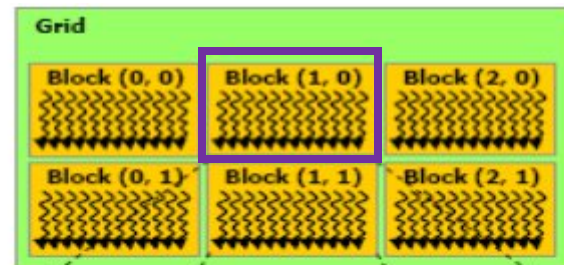
GPU

Threads hierarchy

GPU

GPU

- ❖ Let's compute the index of the chosen thread
 - ❖ $\text{BlockIdx.x} = 1$; $\text{BlockIdx.y} = 0$
 - ❖ $\text{GridDim.x} = 3$; $\text{GridDim.y} = 2$
 - ❖ $\text{Block_glob_id} = \text{BlockIdx.y} \times \text{GridDim.x} + \text{BlockIdx.x} = 0 \times 3 + 1 = 1$



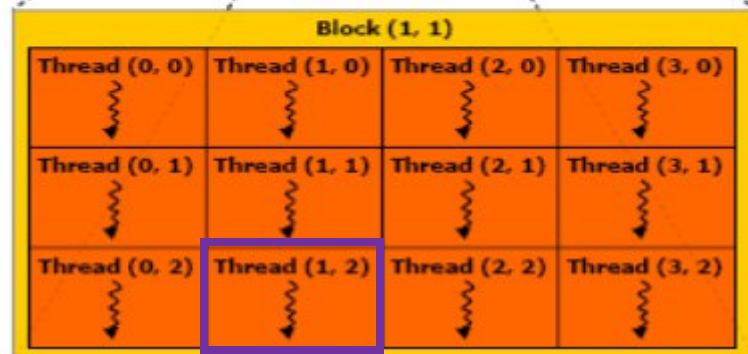
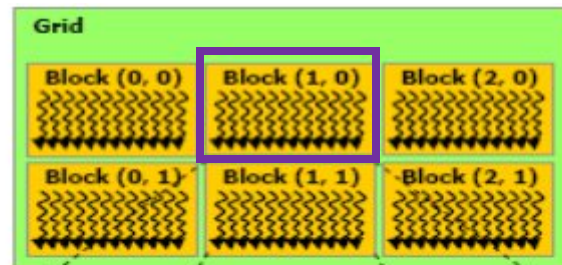
GPU

Threads hierarchy

GPU

GPU

- ❖ Let's compute the index of the chosen thread
 - ❖ $\text{BlockIdx.x} = 1$; $\text{BlockIdx.y} = 0$
 - ❖ $\text{GridDim.x} = 3$; $\text{GridDim.y} = 2$
- ❖ $\text{Block_glob_id} = \text{BlockIdx.y} \times \text{GridDim.x} + \text{BlockIdx.x} = 0 \times 3 + 1 = 1$
 - ❖ $\text{BlockDim.x} = 4$
 - ❖ $\text{BlockDim.y} = 3$



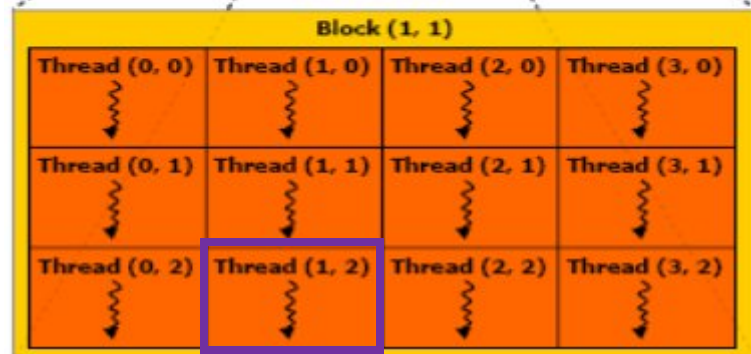
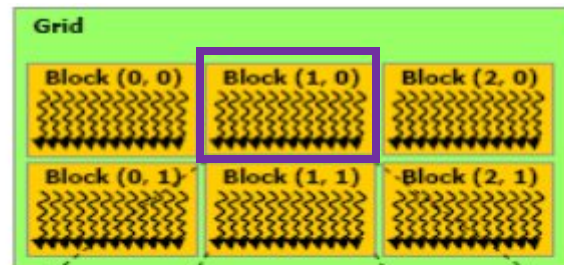
GPU

Threads hierarchy

GPU

GPU

- ❖ Let's compute the index of the chosen thread
 - ❖ $\text{BlockIdx.x} = 1$; $\text{BlockIdx.y} = 0$
 - ❖ $\text{GridDim.x} = 3$; $\text{GridDim.y} = 2$
- ❖ $\text{Block_glob_id} = \text{BlockIdx.y} \times \text{GridDim.x} + \text{BlockIdx.x} = 0 \times 3 + 1 = 1$
- ❖ $\text{BlockDim.x} = 4$
- ❖ $\text{BlockDim.y} = 3$
- ❖ $\text{Nb_threads_per_block} = \text{BlockDim.x} \times \text{BlockDim.y} = 4 \times 3 = 12$



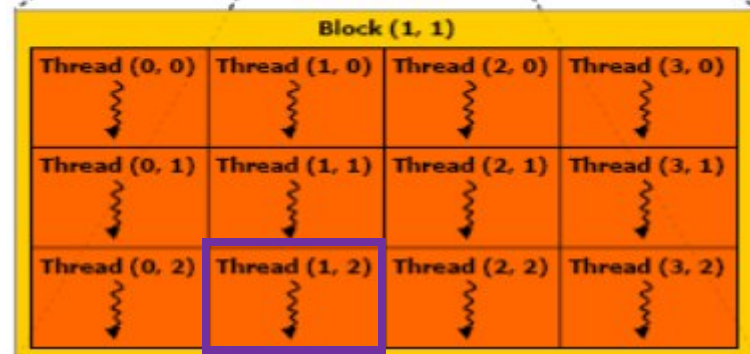
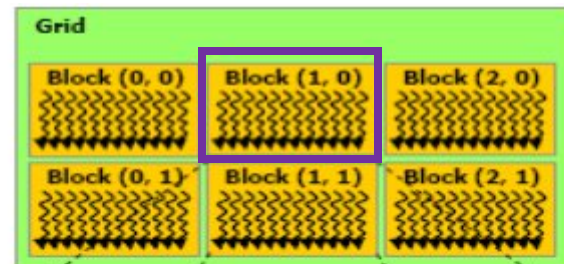
GPU

Threads hierarchy

GPU

GPU

- ❖ Let's compute the index of the chosen thread
 - ❖ $\text{BlockIdx.x} = 1$; $\text{BlockIdx.y} = 0$
 - ❖ $\text{GridDim.x} = 3$; $\text{GridDim.y} = 2$
- ❖ $\text{Block_glob_id} = \text{BlockIdx.y} \times \text{GridDim.x} + \text{BlockIdx.x} = 0 \times 3 + 1 = 1$
- ❖ $\text{BlockDim.x} = 4$
 $\text{BlockDim.y} = 3$
- ❖ $\text{Nb_threads_per_block} = \text{BlockDim.x} \times \text{BlockDim.y} = 4 \times 3 = 12$
 - ❖ $\text{threadIdx.x} = 1$
 - ❖ $\text{threadIdx.y} = 2$
 - ❖ $\text{BlockDim.x} = 4$



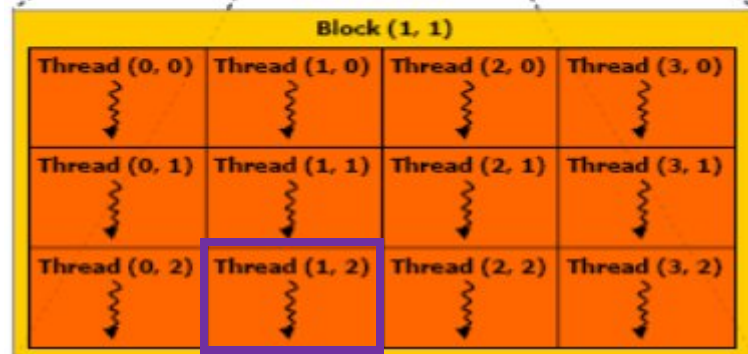
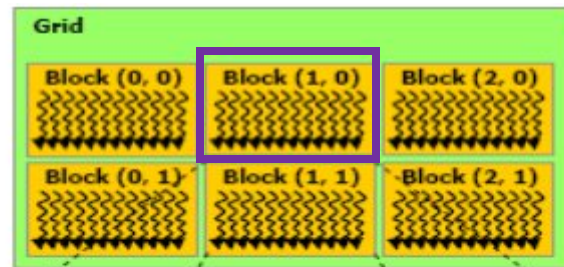
GPU

Threads hierarchy

GPU

GPU

- ❖ Let's compute the index of the chosen thread
 - ❖ $\text{BlockIdx.x} = 1$; $\text{BlockIdx.y} = 0$
 - ❖ $\text{GridDim.x} = 3$; $\text{GridDim.y} = 2$
- ❖ $\text{Block_glob_id} = \text{BlockIdx.y} \times \text{GridDim.x} + \text{BlockIdx.x} = 0 \times 3 + 1 = 1$
- ❖ $\text{BlockDim.x} = 4$
 $\text{BlockDim.y} = 3$
- ❖ $\text{Nb_threads_per_block} = \text{BlockDim.x} \times \text{BlockDim.y} = 4 \times 3 = 12$
- ❖ $\text{threadIdx.x} = 1$
 $\text{threadIdx.y} = 2$
 $\text{BlockDim.x} = 4$
- ❖ $\text{Thread_loc_id} = \text{threadIdx.y} \times \text{BlockDim.x} + \text{threadIdx.x} = 2 \times 4 + 1 = 9$



GPU

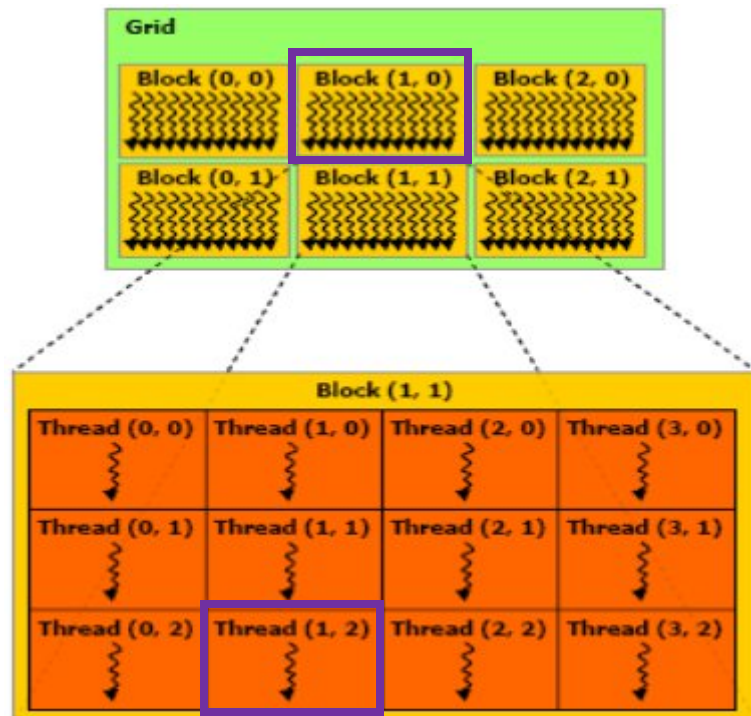
Threads hierarchy

GPU

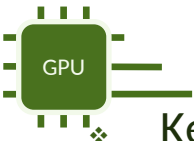
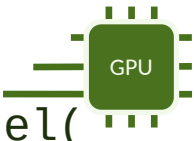
GPU

- ❖ Let's compute the index of the chosen thread
 - ❖ $\text{BlockIdx.x} = 1$; $\text{BlockIdx.y} = 0$
 - ❖ $\text{GridDim.x} = 3$; $\text{GridDim.y} = 2$
- ❖ $\text{Block_glob_id} = \text{BlockIdx.y} \times \text{GridDim.x} + \text{BlockIdx.x} = 0 \times 3 + 1 = 1$
 - ❖ $\text{BlockDim.x} = 4$
 - ❖ $\text{BlockDim.y} = 3$
- ❖ $\text{Nb_threads_per_block} = \text{BlockDim.x} \times \text{BlockDim.y} = 4 \times 3 = 12$
 - ❖ $\text{threadIdx.x} = 1$
 - ❖ $\text{threadIdx.y} = 2$
 - ❖ $\text{BlockDim.x} = 4$
- ❖ $\text{Thread_loc_id} = \text{threadIdx.y} \times \text{BlockDim.x} + \text{threadIdx.x} = 2 \times 4 + 1 = 9$
- ❖ $\text{Thread_glob_id} = \text{Block_glob_id} \times \text{Nb_threads_per_block} + \text{Thread_loc_id} = 1 \times 12 + 9 = 21$

GPU



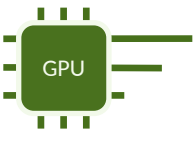
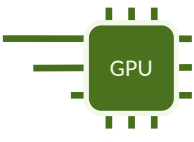
CUDA programming

GPUGPU

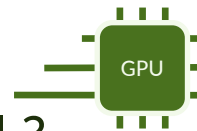
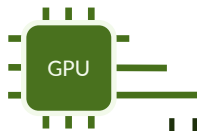
Kernel declaration

- ❖ Function attribute : `__global__`
- ❖ Compute thread index depending on the grid and blocks
 - ❖ Example on the right with a 1D grid and 1D blocks
- ❖ We add a test, checking the index is not greater than the array size
 - ❖ Correspond to the boundary of the original loop

```
__global__ void vecAddKernel(  
    double *a,  
    double *b,  
    double *c, int N) {  
  
    int i ;  
  
    i = blockIdx.x * blockDim.x  
    + threadIdx.x ;  
  
    if ( i < N ) {  
        c[i] = a[i] + b[i];  
    }  
}
```

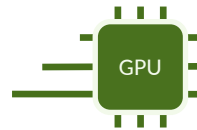
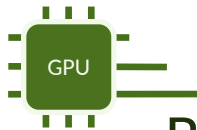
GPUGPU

Kernels invocation



- ❖ How to choose the sizes of the grid and the blocks in the kernel ?
- ❖ The user specifies the sizes at kernel invocation
 - ❖ Several ways to do it
- ❖ Most common syntax of kernel invocation on the *device*
`my_kernel<<<Dg, Db>>>(arg1, arg2, arg3 );`
 - ❖ Dg : grid size (e.i., number of blocks in each dimension) (type dim3)
 - ❖ Db : block size (e.i., number of threads in each dimension) (type dim3)
- ❖ Total number of blocks ?
- ❖ Thread number per block ?
- ❖ Total number of threads ?

Kernels invocation



- ❖ Blackwell introduced a new level
- ❖ Thread block cluster
 - ❖ Between GRID and BLOCK
- ❖ Not possible to integrate that into `<<<Dg, Db>>>`
 - ❖ Compatibility issue
- ❖ compile-time kernel attribute
 - ❖ `__cluster_dims__(X,Y,Z)`
 - ❖ or use `cudaLaunchKernelEx`

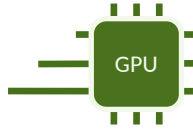
Complete CUDA programm

HOST

```
void vectAdd(  
    double * a, double * b, double * c, int N) {  
  
    double * d_a ;  
    double * d_b ;  
    double * d_c ;  
  
    cudaMalloc((void **)&d_a, N*sizeof(double));  
    cudaMalloc((void **)&d_b, N*sizeof(double));  
    cudaMalloc((void **)&d_c, N*sizeof(double));  
  
    cudaMemcpy(d_a, a, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_c, c, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
  
    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);  
  
    cudaMemcpy(c, d_c, N*sizeof(double),  
        cudaMemcpyDeviceToHost);  
  
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);  
}
```

DEVICE

```
__global__ void vecAddKernel( double  
    *a, double *b, double *c, int N) {  
  
    int i ;  
  
    i = blockIdx.x * blockDim.x +  
        threadIdx.x ;  
  
    if ( i<N ) {  
        c[i] = a[i]+b[i];  
    }  
}
```



Programme CUDA complet

HOST

```
void vectAdd(
    double * a, double * b, double * c, int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMalloc((void **)&d_a, N*sizeof(double));
    cudaMalloc((void **)&d_b, N*sizeof(double));
    cudaMalloc((void **)&d_c, N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
               cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
               cudaMemcpyHostToDevice);

    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
               cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

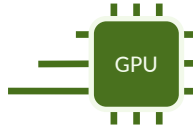
DEVICE

```
__global__ void vecAddKernel( double
    *a, double *b, double *c, int N) {

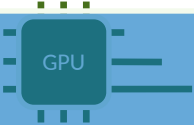
    int i ;

    i = blockIdx.x * blockDim.x +
        threadIdx.x ;

    if ( i<N ) {
        c[i] = a[i]+b[i];
    }
}
```



Programme CUDA complet



HOST

```
void vectAdd(
    double * a, double * b, double * c, int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

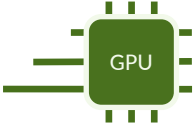
    cudaMalloc((void **)&d_a, N*sizeof(double));
    cudaMalloc((void **)&d_b, N*sizeof(double));
    cudaMalloc((void **)&d_c, N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
               cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
               cudaMemcpyHostToDevice);

    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
               cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```



DEVICE

```
__global__ void vecAddKernel( double
    *a, double *b, double *c, int N) {

    int i ;

    i = blockIdx.x * blockDim.x +
        threadIdx.x ;

    if ( i<N ) {
        c[i] = a[i]+b[i];
    }
}
```

Programme CUDA complet

HOST

```
void vectAdd(
    double * a, double * b, double * c, int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMalloc((void **)&d_a, N*sizeof(double));
    cudaMalloc((void **)&d_b, N*sizeof(double));
    cudaMalloc((void **)&d_c, N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
        cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
        cudaMemcpyHostToDevice);

    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
        cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

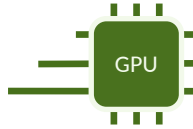
DEVICE

```
__global__ void vecAddKernel( double
    *a, double *b, double *c, int N) {

    int i ;

    i = blockIdx.x * blockDim.x +
        threadIdx.x ;

    if ( i<N ) {
        c[i] = a[i]+b[i];
    }
}
```



Programme CUDA complet

HOST

```
void vectAdd(
    double * a, double * b, double * c, int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMalloc((void **)&d_a, N*sizeof(double));
    cudaMalloc((void **)&d_b, N*sizeof(double));
    cudaMalloc((void **)&d_c, N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
        cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
        cudaMemcpyHostToDevice);

    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
        cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

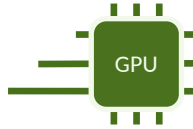
DEVICE

```
__global__ void vecAddKernel( double
    *a, double *b, double *c, int N) {

    int i ;

    i = blockIdx.x * blockDim.x +
        threadIdx.x ;

    if ( i<N ) {
        c[i] = a[i]+b[i];
    }
}
```



Programme CUDA complet

HOST

```
void vectAdd(
    double * a, double * b, double * c, int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMalloc((void **)&d_a, N*sizeof(double));
    cudaMalloc((void **)&d_b, N*sizeof(double));
    cudaMalloc((void **)&d_c, N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
        cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
        cudaMemcpyHostToDevice);

    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
        cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

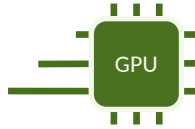
DEVICE

```
__global__ void vecAddKernel( double
    *a, double *b, double *c, int N) {

    int i ;

    i = blockIdx.x * blockDim.x +
        threadIdx.x ;

    if ( i<N ) {
        c[i] = a[i]+b[i];
    }
}
```



Programme CUDA complet

HOST

```
void vectAdd(
    double * a, double * b, double * c, int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMalloc((void **)&d_a, N*sizeof(double));
    cudaMalloc((void **)&d_b, N*sizeof(double));
    cudaMalloc((void **)&d_c, N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
               cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
               cudaMemcpyHostToDevice);

    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
               cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

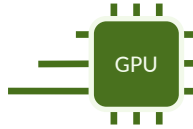
DEVICE

```
__global__ void vecAddKernel( double
    *a, double *b, double *c, int N) {

    int i ;

    i = blockIdx.x * blockDim.x +
        threadIdx.x ;

    if ( i<N ) {
        c[i] = a[i]+b[i];
    }
}
```



Programme CUDA complet

HOST

```
void vectAdd(
    double * a, double * b, double * c, int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMalloc((void **)&d_a, N*sizeof(double));
    cudaMalloc((void **)&d_b, N*sizeof(double));
    cudaMalloc((void **)&d_c, N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
        cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
        cudaMemcpyHostToDevice);

    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
        cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

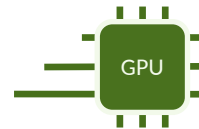
DEVICE

```
__global__ void vecAddKernel( double
    *a, double *b, double *c, int N) {

    int i ;

    i = blockIdx.x * blockDim.x +
        threadIdx.x ;

    if ( i<N ) {
        c[i] = a[i]+b[i];
    }
}
```



Programme CUDA complet

HOST

```
void vectAdd(
    double * a, double * b, double * c, int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMalloc((void **)&d_a, N*sizeof(double));
    cudaMalloc((void **)&d_b, N*sizeof(double));
    cudaMalloc((void **)&d_c, N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
        cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
        cudaMemcpyHostToDevice);

    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
        cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

DEVICE

```
__global__ void vecAddKernel( double
    *a, double *b, double *c, int N) {

    int i ;

    i = blockIdx.x * blockDim.x +
        threadIdx.x ;

    if ( i<N ) {
        c[i] = a[i]+b[i];
    }
}
```

